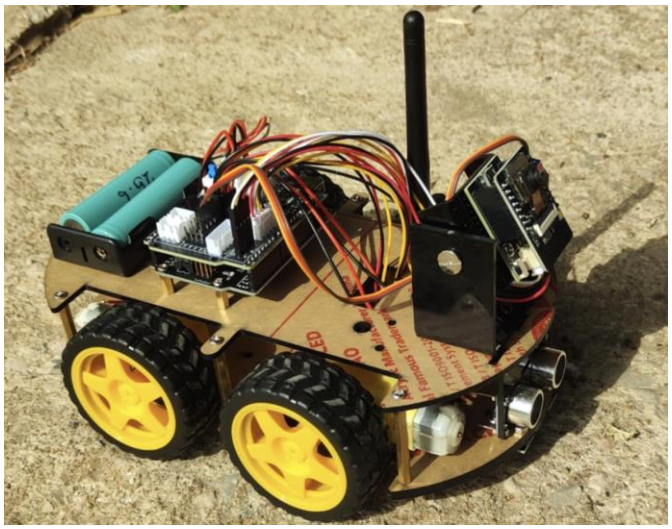
การเขียนโปรแกรมควบคุมรถหุ่นยนต์อัจฉริยะ (Smart Robot Car)

ในการอบรมการเขียนโปรแกรมควบคุมรถหุ่นยนต์อัจฉริยะครั้งนี้จะใช้รถหุ่นยนต์ที่มีความสามารถและลักษณะเฉพาะดังนี้คือ

1. มีล้อ 4 ล้อโดยที่แต่ละล้อสามารถหมุนไปหน้าและถอยหลังได้อย่างอิสระ ซึ่งจะสามารถทำให้ตัวหุ่นยนต์เคลื่อนที่ไปในทิศทางต่าง ๆ ได้ โดยการสั่งให้แต่ละล้อหมุนไปในทิศทางที่ต้องการ
2. มีเซ็นเซอร์สำหรับการตรวจจับพื้นที่มีสีแตกต่างกันโดยแสงอินฟาเรด เช่น ระหว่างพื้นสีเข้มและพื้นสีอ่อนได้ โดยมีเซ็นเซอร์อยู่จำนวน 3 ตัว ดังนั้นสามารถเขียนโปรแกรมให้หุ่นยนต์สามารถเดินตามเส้นที่มีสีแตกต่างจากพื้นสีอ่อนได้
3. มีเซ็นเซอร์แบบอัลตราโซนิค (Ultrasonic) จึงสามารถตรวจจับสิ่งกีดขวางด้านหน้าของตัวหุ่นยนต์ได้ และสามารถเขียนโปรแกรมให้หลบหลีกสิ่งกีดขวางได้
4. มีกล้องวิดีโอด้านหน้าของหุ่นยนต์ สามารถถ่ายทอดภาพที่เป็นภาพเคลื่อนไหวส่งกลับมายังต้นทางได้ โดยใช้ตัว ESP32-CAM เป็นตัวควบคุมกล้อง
5. เนื่องจากใช้ตัว ESP32-CAM เป็นตัวควบคุมกล้อง จึงสามารถอาศัยความสามารถของ CHIP ESP32 ในการติดต่อผ่านระบบ WiFi ได้ ไม่ว่าจะเป็น Access Point Mode หรือ Infrastructure Mode
6. ควบคุมด้วยรีโมทคอลโทรล (Remote Control) ด้วยแสงอินฟราเรด (แต่ส่วนนี้จะไม่ได้มีการสอนเนื่องจากข้อจำกัดในด้านเวลา)



หลังจากจบการอบรมครั้งนี้คาดว่าผู้เข้าร่วมอบรมจะสามารถเข้าใจและมีความรู้ความสามารถดังนี้

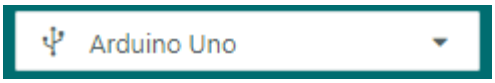
1. สามารถเขียนโปรแกรมควบคุมบอร์ด Arduino Uno ผ่าน Arduino IDE ได้
2. สามารถเขียนโปรแกรมควบคุมล้อรถหุ่นยนต์ให้เคลื่อนที่ในทิศทางต่าง ๆ ได้
3. สามารถเขียนโปรแกรมเพื่อใช้งานเซ็นเซอร์ตรวจจับความเข้มของพื้น และควบคุมให้รถหุ่นยนต์เดินตามเส้นที่กำหนดได้
4. สามารถเขียนโปรแกรมเพื่อใช้งานเซ็นเซอร์แบบอัลตราโซนิค และสามารถควบคุมรถหุ่นยนต์ให้เดินหลบสิ่งกีดขวางได้
5. สามารถปรับปรุงโปรแกรมเพื่อรับภาพวิดีโอจากกล้องและส่งกลับมายังโทรศัพท์มือถือหรือคอมพิวเตอร์
6. สามารถปรับปรุงโปรแกรมเพื่อเชื่อมต่อกับรถยนต์หุ่นยนต์ผ่าน WiFi ใน Access Point Mode และ Infrastructure Mode เพื่อควบคุมรถหุ่นยนต์แบบไร้สายได้

การใช้งาน Arduino IDE

Arduino เป็นชื่อของแพลตฟอร์มแบบเปิดเผยซอร์สโค้ด (Open Source) ที่สร้างขึ้นเพื่อให้สามารถใช้งานได้ง่าย โดยจะประกอบไปด้วยบอร์ดไมโครคอนโทรลเลอร์ที่มีราคาย่อมเยา และซอฟต์แวร์ที่ใช้ในการเขียนโปรแกรมที่เรียกว่า Arduino IDE (Arduino Integrated Development Environment)

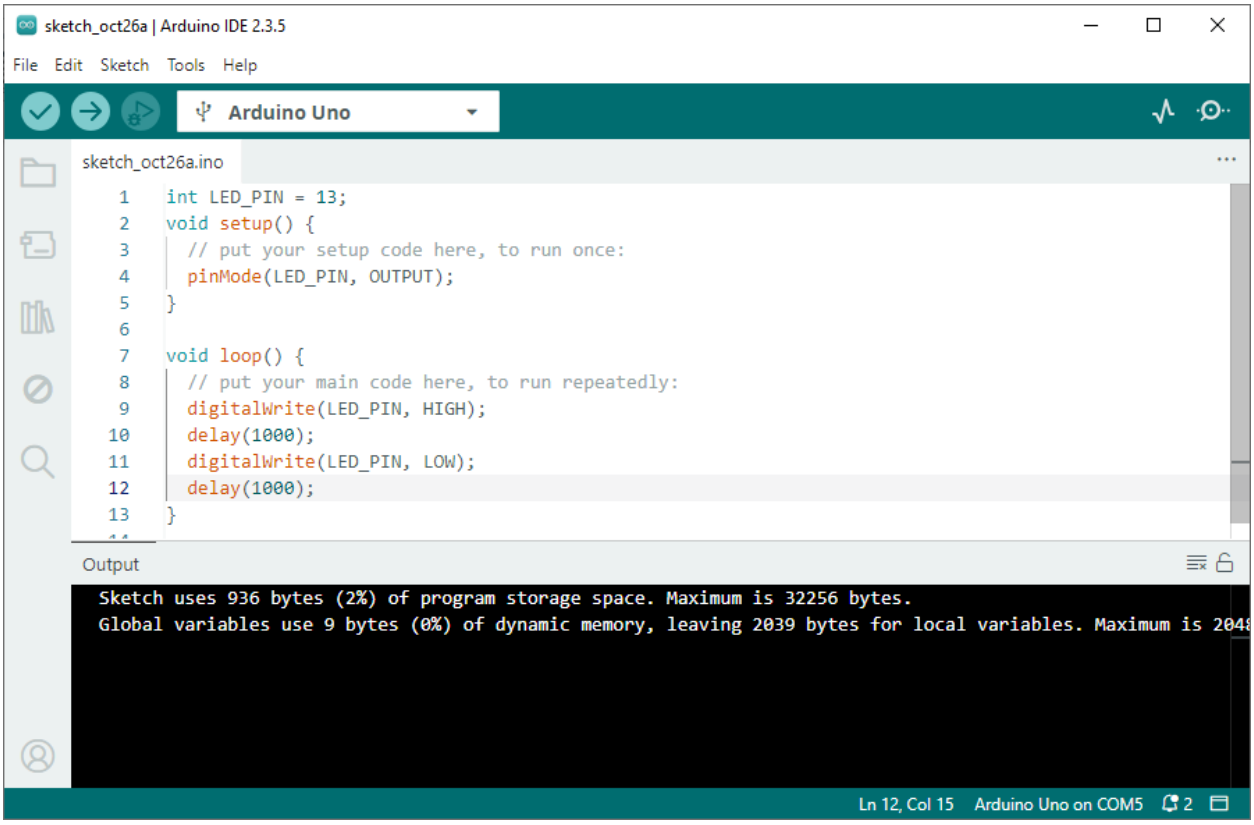
โดยในรถหุ่นยนต์ที่จะให้ในการอบรมครั้งนี้จะมีบอร์ดไมโครคอนโทรลเลอร์ในการควบคุมสิ่งต่าง ๆ บนรถหุ่นยนต์ มีชื่อบอร์ดว่า Arduino Uno และ ESP32-CAM โดยภาษาที่ใช้จะเป็นภาษา C++ แต่อาศัย Library ที่ชื่อว่า Arduino ช่วย จึงทำให้การเขียนโปรแกรมได้ง่ายและสะดวกมากขึ้น เหมาะสำหรับบุคคลทั่วไปที่ไม่ได้มีความเชี่ยวชาญด้านการเขียนโปรแกรมและความรู้ทางด้านอิเล็กทรอนิกส์มากนัก

Arduino IDE จะมีความซับซ้อนน้อย คือตัวโปรแกรมมีพื้นที่สำหรับเขียนโปรแกรม และมีปุ่มหลัก ๆ สำหรับใช้งานไม่กี่ปุ่มเท่านั้น เช่นปุ่มสำหรับ Verify  และปุ่มสำหรับ Upload  ส่วนปุ่มอื่นแทบไม่เคยต้องใช้งานเลย โดยปกติแล้วถ้ายังไม่ได้เชื่อมต่อบอร์ด Arduino Uno ผ่านทางช่อง USB ของคอมพิวเตอร์ เราจะไม่สามารถ Upload โปรแกรมได้ โดยให้สังเกตจากช่องที่แสดงว่า Arduino IDE ว่ากำลังต่ออยู่กับบอร์ดอะไร ถ้า

ยังไม่ได้เชื่อมต่อ ตัวอักษรในช่องตามภาพจะเป็นตัวบางแบบธรรมดา 

แต่เมื่อเชื่อมต่อสาย USB เข้ากับบอร์ด Arduino Uno แล้ว ตัวอักษรในช่องที่ระบุว่า Arduino Uno ก็จะเปลี่ยนเป็นตัวหนาดังภาพ 

นั่นคือพร้อมที่จะรับการ Upload โปรแกรมที่เขียนขึ้นแล้ว ภาพด้านล่างคือภาพเต็มของหน้าจอของโปรแกรม Arduino IDE พร้อมด้วยโปรแกรมสำหรับการสั่งให้ LED บนบอร์ดกระพริบ โดยการติด 1 วินาทีสลับกับดับ 1 วินาทีสลับการไปเรื่อย ๆ จนกว่าจะมีการถอดปลั๊ก USB



อธิบายการทำงานของโปรแกรมโดยสังเขปคือในบรรทัดที่ 1 เป็นการกำหนดว่า LED ใน digital pin หมายเลข 13 จะถูกให้สำหรับการแสดงการกระพริบของหลอด LED ที่มีอยู่บนบอร์ด Arduino Uno จากโรงงาน ส่วนฟังก์ชันในโปรแกรมที่มีชื่อว่า setup ในช่วงบรรทัดที่ 2-5 นั้นใช้สำหรับการตั้งค่าเริ่มต้นให้กับโปรแกรม หรือสิ่งที่ต้องการให้ทำครั้งเดียวตอนเริ่มโปรแกรม ก็จะเขียนไว้ในฟังก์ชันนี้ ส่วนบรรทัดที่ 4 เป็นการกำหนดให้ LED_PIN หรือ digital pin หมายเลข 13 ให้เป็น OUTPUT โดยใช้คำสั่ง pinMode แปลได้ว่าจะส่งกระแสไฟฟ้าออกจาก pin เมื่อถูกสั่งให้เป็น HIGH นั่นเอง โดยปกติแล้ว digital pin สามารถกำหนดให้เป็น INPUT หรือ

OUTPUT ก็ได้โดยที่ INPUT หมายถึงรับกระแสไฟฟ้าเข้าไปทาง digital pin เพื่อให้ทำงานบางอย่าง ส่วน OUTPUT หมายถึงส่งกระแสไฟฟ้าออกจาก pin เพื่อไปสั่งงานบางอย่าง และในโปรแกรมตัวอย่างนี้เป็นการจ่ายไฟฟ้าออกทาง pin 13 เพื่อเป็นพลังงานให้กับหลอด LED นั่นเอง

ในบรรทัดที่ 7-13 คือฟังก์ชันที่ชื่อ loop แปลว่าฟังก์ชันนี้จะทำงานไปเรื่อย ๆ ตราบใดที่ยังมีกระแสไฟฟ้าเลี้ยงตัวบอร์ด ดังนั้นโปรแกรมก็จะทำงานไปเรื่อย ๆ จนกว่าจะถูกตัดกระแสไฟฟ้าหรือบอร์ดเสียสาย ในบรรทัดที่ 9 คำสั่ง digitalWrite เป็นการสั่งให้ LED_PIN มีสถานะเป็น HIGH หรือจ่ายกระแสไฟฟ้าออกไป โดยกระแสที่จ่ายได้ไม่เกิน 20mA ที่ 5V เท่านั้น ถือว่าน้อยมาก แต่ก็เพียงพอสำหรับทำให้หลอด LED สว่างได้ คำสั่งในบรรทัดที่ 10 เป็นการหน่วงเวลา 1000 มิลลิวินาที หรือเท่ากับ 1 วินาทีนั่นเอง และเมื่อครบ 1000 มิลลิวินาที คำสั่ง digitalWrite ในบรรทัดที่ 11 ก็จะทำให้ LED_PIN เป็น LOW หรือตัดการจ่ายกระแสไฟฟ้านั่นเอง เป็นเหตุให้ LED ดับ และหน่วงการดับเอาไว้อีก 1 วินาที ในบรรทัดที่ 12 จากนั้นก็จะวนกลับไปทำในบรรทัดที่ 9 ใหม่ วนไปเรื่อย ๆ แบบไม่รู้จบ

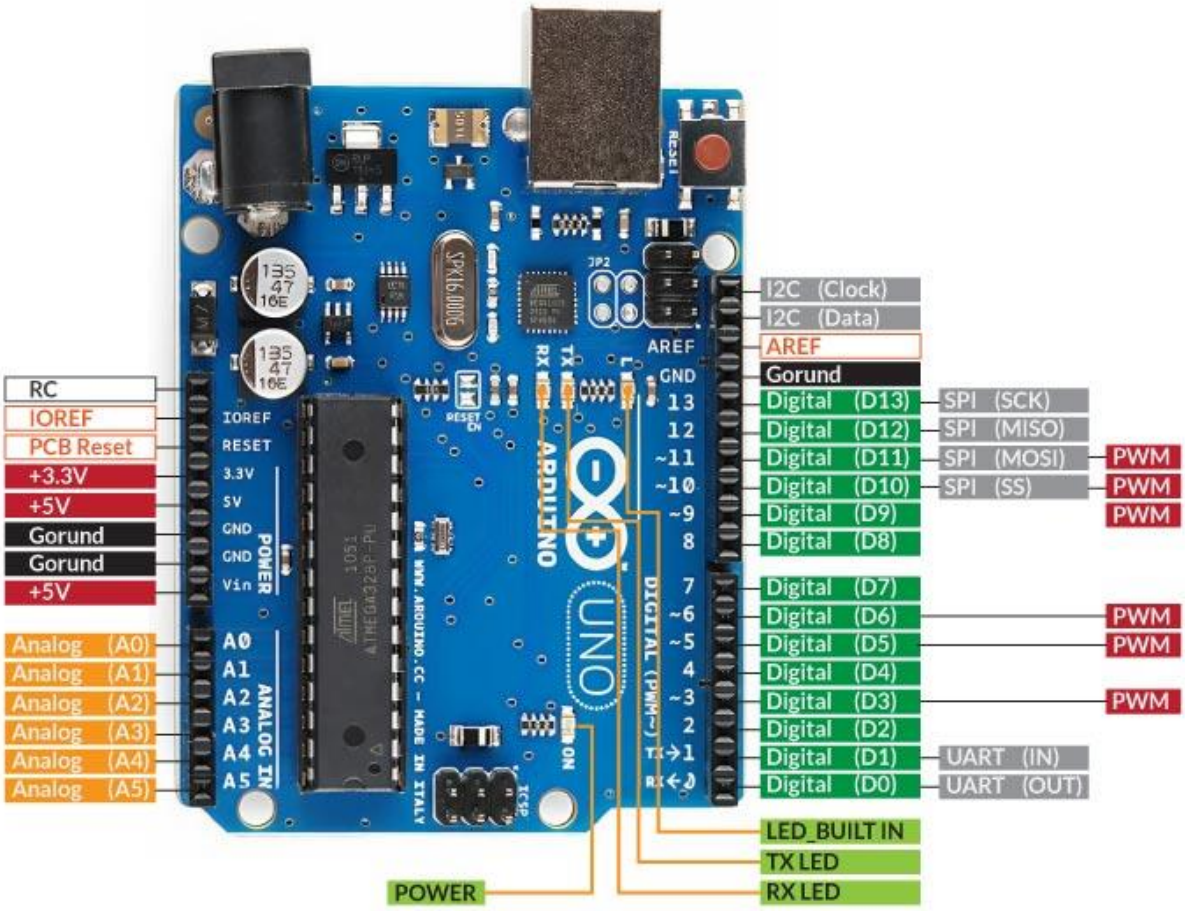
ด้านล่างนี้เป็นภาพบอร์ด Arduino Uno ซึ่งโดยปกติแล้วหน้าตาจะเป็นแบบนี้ หลาย ๆ บริษัทที่ผลิตออกมาหน้าตาจะคล้ายกันหมด แต่อาจจะแตกต่างกันตรงสีหรือ CHIP บนบอร์ดที่อาจเป็นคนละรุ่นกันก็ได้



ที่มา: https://components101.com/sites/default/files/components/Arduino-Uno-Board.jpg

ภาพต่อไปเป็นภาพของ pin หรือขาต่าง ๆ ของบอร์ด Arduino Uno ซึ่งจะหลัก ๆ จะเป็นขาไฟเลี้ยงต่าง ๆ ขาที่เป็นแบบ digital จากหมายเลข 0 – 13 และ Analog A0-A5

อธิบายแบบกระชับคือ digital pin ถ้าตั้งให้เป็น OUTPUT เมื่อถูกสั่งด้วยคำสั่ง digitalWrite ให้เป็น HIGH บอร์ดก็จะจ่ายไฟฟ้ากระแสไม่เกิน 20mA/5V ออกไปทาง pin ที่กำหนด แต่เมื่อสั่งให้เป็น LOW ก็จะตัดการจ่ายกระแสไฟฟ้านั่นเอง และถ้าถูกกำหนดให้เป็น INPUT ถ้ามีการป้อนไฟฟ้า 3-5V เข้าไปทาง pin ดังกล่าว บอร์ดก็อ่านค่าได้เป็น HIGH ผ่านคำสั่ง digitalRead แต่ถ้าไม่ได้จ่ายกระแสไฟฟ้าเข้าไปหรือมีไฟฟ้าแรงดันไม่เกิน 3V บอร์ดก็จะอ่านค่าเป็น LOW ผ่านคำสั่ง digitalRead เช่นกัน



ที่มา: https://components101.com/sites/default/files/component_pin/Arduino-UNO-Parts-Marking.jpg

ส่วน Analog pin หมายเลข A0-A5 สามารถทำเป็น INPUT ได้เท่านั้น ไม่สามารถเป็น OUTPUT ได้ แปลว่าสามารถรับค่าได้ 0-5V โดยที่แรงดัน 0V จะถูกแปลงเป็น 0 โดยบอร์ด Arduino ผ่านคำสั่ง analogRead และถ้า 2.5V จะเป็น 511 ส่วนแรงดัน 5V จะเป็นค่า 1023 ส่วนค่าอื่น ๆ ระหว่าง 0-5V สามารถเทียบบัญญัติไตรยางค์เอาโดยประมาณได้

จะสังเกตได้ว่า digital จะมีทั้ง INPUT และ OUTPUT แต่ analog จะมีเพียง INPUT เท่านั้น โดยที่ anolog ที่เป็น OUTPUT สามารถทำได้ผ่าน digital pin โดยคำสั่งที่ชื่อว่า analogWrite โดยอาศัยความสามารถของ PWM (Pulse Width Modulate) เป็นตัวช่วย หลักการโดยสังเขปมีดังนี้ ในเมื่อไม่สามารถจ่ายไฟฟ้าให้ค่าแรงดันที่อยู่ระหว่าง 0 – 5V ได้ เพราะบอร์ด Arduino สามารถจ่ายแรงดันไฟฟ้าได้เพียง 2 ค่าคือ 0V หรือ 5V แต่ถ้าเราจำเป็นจะต้องจ่าย 2.5V จะทำได้อย่างไร คำตอบคือทำไม่ได้ แต่เรามีวิธีที่เทียบเอาได้โดยอาศัยเวลาในการจ่ายไฟฟ้า กล่าวคือสมมติว่าเราจ่ายไฟฟ้าเป็นเวลา 1 วินาที แทนที่เราจะจ่ายทั้ง 1 วินาที ก็จะจ่ายแค่ 0.5 วินาที ส่วนอีก 0.5 วินาทีก็หยุดจ่ายไฟ ดังนั้นแรงดันเฉลี่ยก็ประมาณเอาว่าเป็น 2.5V การทำแบบนี้เรียกว่า PWM จากตัวอย่างดังกล่าวมาคือการทำ PWM มี duty cycle 50% นั่นเอง แต่ในการใช้งานจริง ๆ ถ้าทำแบบนั้นก็จะพบว่าจ่ายกระแสไฟฟ้าเป็นช่วง ๆ แทนที่จะได้ 2.5V ตามที่ต้องการ ดังนั้นเมื่อใช้งานจริง ๆ ต้องทำแบบมีความเร็วในการสลับที่สูงมาก ๆ เช่น 1 วินาที สลับไป 10000 ครั้งหรือ 10000Hz นั่นเอง ซึ่งความถี่ระดับนั้นสายตามนุษย์เราไม่สามารถมองเห็นได้

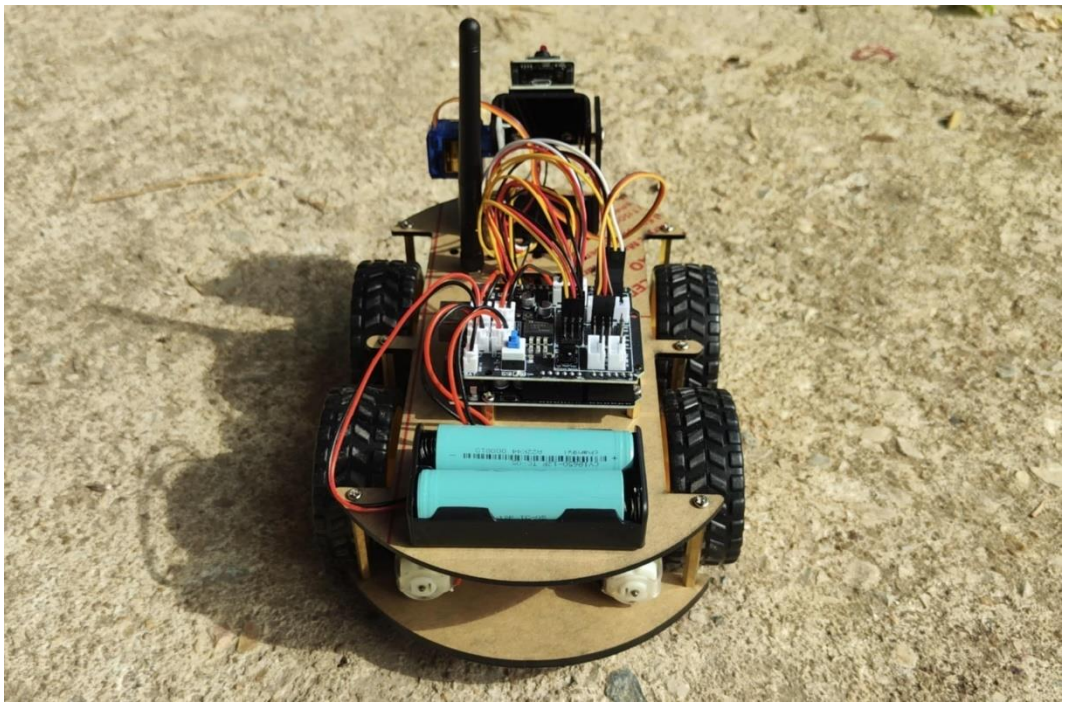
จากที่กล่าวมาทั้งหมดข้างบน น่าจะเป็นพื้นฐานความเข้าใจให้กับการเขียนโปรแกรมสั่งรถหุ่นยนต์ให้ทำงานได้แล้ว ในบทเรียนหลังจากนี้จะเป็นเริ่มเข้าสู่การเขียนโปรแกรมควบคุมรถหุ่นยนต์

1. การควบคุมล้อรถหุ่นยนต์

รถหุ่นยนต์มีล้อจำนวน 4 ล้อ โดยที่แต่ละล้อจะมีมอเตอร์คุมตัวละล้อ ดังนั้นทำให้แต่ละล้อหมุนได้อย่างอิสระทั้งทิศทางและความเร็วในการหมุน การหมุนทั้ง 4 ล้อมีลักษณะดังนี้

- ล้อทั้ง 4 ล้อหมุนไปด้านหน้า จะทำให้รถหุ่นยนต์วิ่งไปด้านหน้า
- ล้อทั้ง 4 ล้อหมุนไปด้านหลัง จะให้ให้รถหุ่นยนต์วิ่งถอยหลัง
- 2 ล้อซ้ายหยุดหมุนและ 2 ล้อขวาหมุนไปด้านหน้า จะทำให้รถหุ่นยนต์เลี้ยวซ้าย
- 2 ล้อขวาหยุดหมุนและ 2 ล้อซ้ายหมุนไปด้านหน้า จะทำให้รถหุ่นยนต์เขี้ยวขวา
- 2 ล้อซ้ายหยุดหมุนและ 2 ล้อขวาหมุนไปด้านหลัง จะทำให้รถหุ่นยนต์เลี้ยวขวา

- 2 ล้อขวาหยุดหมุนและ 2 ล้อซ้ายหมุนไปด้านหลัง จะทำให้รถหุ่นยนต์เขี้ยวซ้าย
- 2 ล้อซ้ายหมุนถอยหลัง และ 2 ล้อขวาหมุนไปด้านหน้า จะทำให้รถหุ่นยนต์หมุนไปทางซ้าย
- 2 ล้อซ้ายหมุนไปด้านหน้า และ 2 ล้อขวาหมุนถอยหลัง จะทำให้รถหุ่นยนต์หมุนไปทางขวา



แต่จากที่กล่าวมาแล้ว กระแสไฟฟ้าที่บอร์ด Arduino Uno จ่ายออกมานั้นมีไม่เกิน 20mA / 5V หรือ 0.02A เท่านั้น แต่มอเตอร์ที่หมุนล้อนั้นกินกระแสไฟฟ้ามากกว่านั้นมาก ดังนั้นจึงจำเป็นต้องมีการเพิ่มกระแสไฟฟ้าให้เพียงพอ โดยอาศัยอุปกรณ์อิเล็กทรอนิกส์อย่างทรานซิสเตอร์มาทำหน้าที่ ในรถหุ่นยนต์ด้านบนบอร์ด Arduino Uno นั้นมีอีกบอร์ดซ้อนอยู่ เรียกกันว่า Shield ซึ่งบอร์ดตัวนี้มีบางส่วนทำหน้าที่ขยายกระแสไฟฟ้าให้เพียงพอสำหรับขับมอเตอร์แต่ละล้อ และจับคู่ล้อด้านซ้ายหน้าและหลังเข้าด้วยกัน รวมถึงล้อหน้าขวาและหลังขวาเข้าด้วยกันด้วย เพราะในการควบคุมหุ่นยนต์ที่ใช้ล้อแบบธรรมดาแบบนี้ ล้อข้างเดียวกันจะหมุนไปทางเดียวกันเสมอ

ดังนั้นมีการกำหนดไว้ว่า digital pin หมายเลข 2 คือทิศทางของล้อซ้ายทั้ง 2 ล้อ โดยถ้ามีค่าเป็น HIGH คือหมุนไปด้านหน้า และถ้ามีค่าเป็น LOW คือถอยหลัง ส่วน digital pin หมายเลข 5 คือกำหนดความเร็วของล้อ โดยจะมีค่า 0 – 255 โดยที่ 255 คือมีความเร็วสูงสุด ตรงนี้คือการใช้ PWM มาควบคุมความเร็วล้อนั่นเอง

ในทำนองเดียวกันล้อด้านขวากำหนด digital pin หมายเลข 4 คือทิศทางการหมุนของล้อขวาทั้งสอง แต่จะสลับกับล้อซ้ายคือ LOW เป็นไปหมุนหน้าและ HIGH เป็นถอยหลัง และ digital pin หมายเลข 6 คือความเร็วในการหมุน สาเหตุที่สลับกันเพราะล้ออยู่คนละด้าน ทำให้ล้อต้องหมุนสลับกันเพื่อให้หุ่นยนต์วิ่งไปทิศทางเดียวกัน

ดังนั้นในการเขียนโปรแกรมจะต้องกำหนดค่าต่าง ๆ ให้สอดคล้องกับที่ผู้ผลิตรถหุ่นยนต์ได้กำหนดไว้แล้ว ด้านล่างเป็นตัวอย่างโปรแกรมสำหรับทดสอบให้รถหุ่นยนต์วิ่งไปด้านหน้า ด้วยความเร็วล้อซ้ายและขวาไม่เท่ากัน

```
int LEFT_WHEELS = 2;
int LEFT_WHEELS_SPEED = 5;
int RIGHT_WHEELS = 4;
int RIGHT_WHEELS_SPEED = 6;
void setup() {
    pinMode(LEFT_WHEELS, OUTPUT);
    pinMode(LEFT_WHEELS_SPEED, OUTPUT);
    pinMode(RIGHT_WHEELS, OUTPUT);
    pinMode(RIGHT_WHEELS_SPEED, OUTPUT);

    digitalWrite(LEFT_WHEELS, HIGH);
    analogWrite(LEFT_WHEELS_SPEED, 127);
    digitalWrite(RIGHT_WHEELS, LOW);
    analogWrite(RIGHT_WHEELS_SPEED, 200);
}

void loop() {
}
```

จากโปรแกรมด้านบน เมื่อมีการ upload โปรแกรมไปยังรถหุ่นยนต์ จะพบว่าล้อรถยนต์ยังไม่หมุน จำเป็นใส่แบตเตอรี่เข้าไปในรถหุ่นยนต์และเปิดสวิทย์สีฟ้าเสียก่อน ล้อรถหุ่นยนต์จะหมุนไปด้านหน้าทั้งล้อซ้ายและล้อขวา จะเห็นได้ว่าล้อขวาจะหมุนเร็วกว่าด้านซ้าย เพราะล้อซ้ายกำหนดความเร็วไว้ที่ 127 แต่ล้อขวากำหนดไว้ที่ 200 ถ้าให้รถหุ่นยนต์วิ่งบนพื้นก็จะวิ่งไปด้านหน้าและค่อยเลี้ยวโค้งไปทางซ้าย

ข้อสังเกตในฟังก์ชัน loop ไม่มีการเขียนโค้ดใด ๆ เลย เพราะจากโปรแกรมนี้เป็นการสั่งให้รถหุ่นยนต์วิ่งแค่ครั้งเดียวในฟังก์ชัน setup แล้ว แต่ถ้าจะมีการเปลี่ยนความเร็วหรือทิศทางหรือการควบคุมที่ซับซ้อนมากขึ้น ก็จำเป็นต้องเขียนโค้ดไว้ใน loop เพื่อรับค่าจากทาง WiFi หรือ Remote Control ก็ได้ ซึ่งจะกล่าวอีกทีในตอนหลัง

จากโปรแกรมตัวอย่างด้านบนสามารถ เขียนโปรแกรมใหม่ในรูปแบบฟังก์ชันเพื่อความสะดวกในการเรียกใช้ทีหลัง

```
int LEFT_WHEELS = 2;
int LEFT_WHEELS_SPEED = 5;
int RIGHT_WHEELS = 4;
int RIGHT_WHEELS_SPEED = 6;
void car_forward(int left_speed, int right_speed) {
    digitalWrite(LEFT_WHEELS, HIGH);
    analogWrite(LEFT_WHEELS_SPEED, left_speed);
    digitalWrite(RIGHT_WHEELS, LOW);
    analogWrite(RIGHT_WHEELS_SPEED, right_speed);
}
void setup() {
    pinMode(LEFT_WHEELS, OUTPUT);
    pinMode(LEFT_WHEELS_SPEED, OUTPUT);
    pinMode(RIGHT_WHEELS, OUTPUT);
    pinMode(RIGHT_WHEELS_SPEED, OUTPUT);
    car_forward(127, 200);
}

void loop() {
}
```

ให้สังเกตตัวที่ระบายเป็นตัวหนา หลังจากนี้เมื่อต้องการเปลี่ยนความเร็วของการวิ่งไปทางหน้าของรถหุ่นยนต์ ก็สามารถเรียกใช้ฟังก์ชัน car_forward ได้เลย ไม่จำเป็นต้องเขียนโค้ดซ้ำ และสามารถกำหนดความเร็วของล้อแต่ละข้างในวงเล็บด้านหลังของฟังก์ชันดังตัวอย่าง car_forward(127, 200)

2. การให้รถหุ่นยนต์เดินตามเส้น

รถหุ่นยนต์ที่ใช้ในการอบรมครั้งนี้ มีเซ็นเซอร์ด้านหน้าล่างของรถหุ่นยนต์ 3 ตัว คือซ้าย กลาง และขวา โดยตัวซ้ายจะต่อเข้ากับ digital pin 7 ตัวกลาง digital pin 8 และตัวขวาต่อกับ digital pin 9 สามารถทดสอบการทำโดยใช้โค้ดด้านล่าง โดยในการทดสอบครั้งนี้จะใช้คำสั่งเพิ่มเติม คือคำสั่งสำหรับการสั่งพิมพ์ข้อความหรือค่าต่าง ๆ ทาง Serial Monitor คือคำสั่ง Serial.begin(57600) และ Serial.print() โดยปกติแล้วจะใช้ Serial Monitor สำหรับการแสดงค่าต่าง ๆ เพื่อใช้ในการทดสอบหรือการดีบัก (debug) โปรแกรม และโค้ดด้านล่างเป็นตัวอย่างโปรแกรมสำหรับการทดสอบเซ็นเซอร์ตัวซ้าย

```
int LEFT_SENSOR = 7;
void setup() {
  pinMode(LEFT_SENSOR, INPUT_PULLUP);
  Serial.begin(57600);
```

```
}

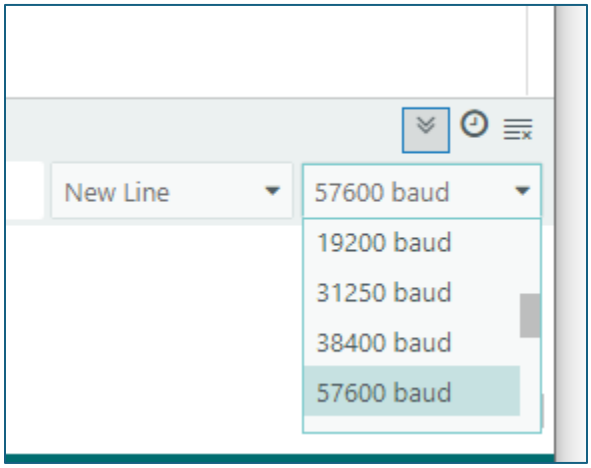
void loop() {
  int left_value = digitalRead(LEFT_SENSOR);
  Serial.println(left_value);
  delay(200);
}
```

ข้อสังเกตแรกตรงคำสั่ง pinMode รอบนี้จะใช้คำว่า INPUT_PULLUP แทนที่จะเป็น INPUT เฉย ๆ เป็นการบอกว่าสถานะของ pin นั้นจะมีค่าเป็น HIGH ตลอดเวลา ยกเซ็นเซอร์สั่งให้เป็น LOW เท่านั้น

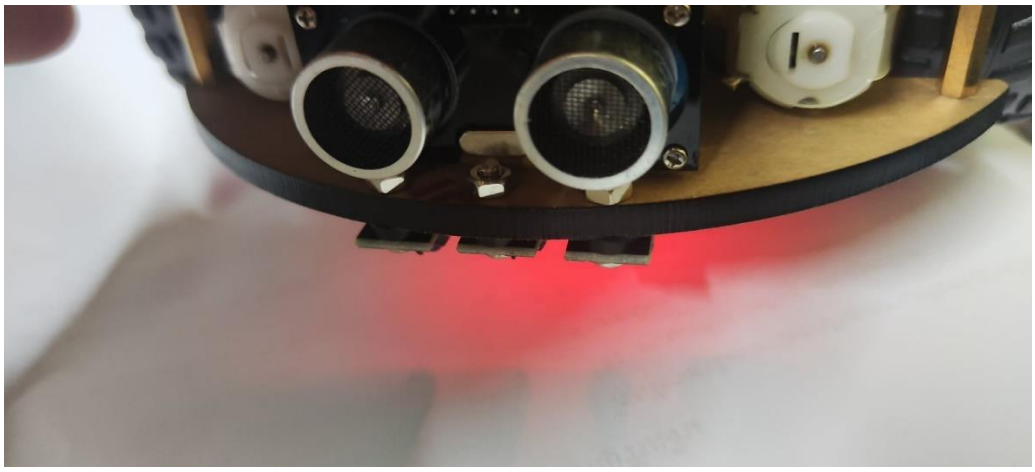
ในฟังก์ชัน setup จะมีบรรทัดที่เขียนว่า Serial.begin(57600) เป็นการกำหนดให้สามารถสื่อสารกับ Serial Monitor ส่วนในฟังก์ชัน loop บรรทัดที่เขียนว่า `Serial.println(left_value)` เป็นการแสดงค่า left_value ออกทาง Monitor โดยให้กดปุ่มด้านขวาบนของ Arduino IDE ดังภาพ เพื่อเปิด Serial Monitor



จากนั้นให้เลือก baud rate เป็น 57600 ให้ตรงกับใน Serial.begin(57600) ดังภาพ เราสามารถเลือกค่าอื่นที่ไม่ใช่ 57600 ก็ได้ แต่ทั้งในฟังก์ชัน Serial.begin() กับใน Serial Monitor จะต้องตรงกัน ถ้าไม่ตรงกันตัวอักษรที่แสดงออกมาจะไม่สามารถอ่านไม่ได้



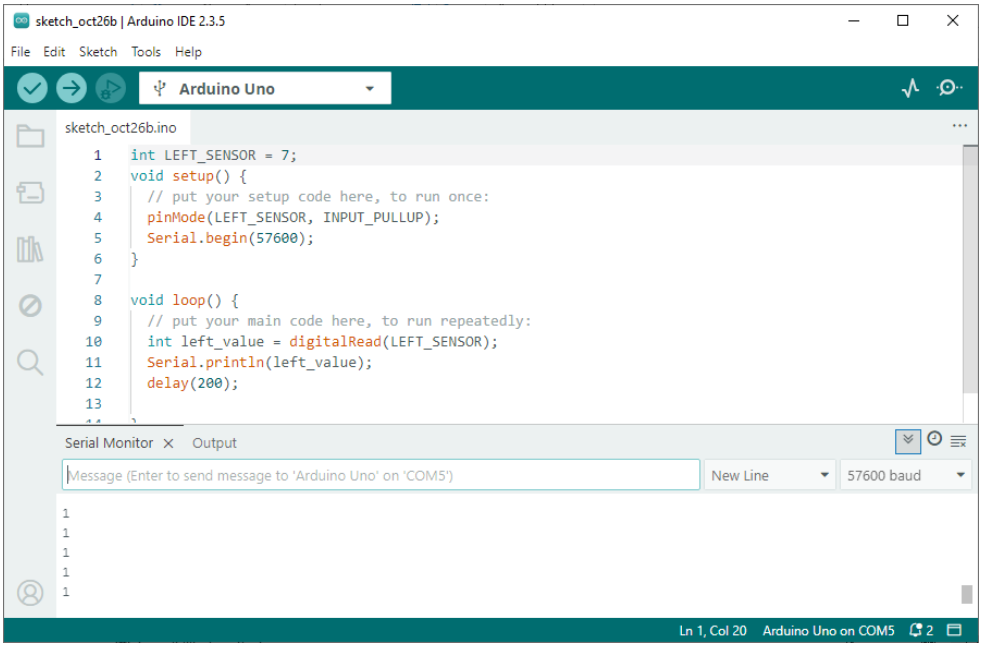
เมื่อ upload โปรแกรมนี้เข้าไปยังรถหุ่นยนต์แล้ว ให้ทดสอบนำกระดาษหรือวัสดุที่มีสีขาวไปอังด้านล่างหน้าของรถหุ่นยนต์ดังภาพ สังเกตจะมีแสงสีแดงที่ตัวเซ็นเซอร์



หลักการทำงานคือเซ็นเซอร์จะส่งแสงอินฟราเรดออกไป เมื่อไปเจอพื้นหรือวัตถุที่มีสีสว่างเช่นพื้นสีขาว จะมีการสะท้อนแสงกลับ ทำให้เซ็นเซอร์รับการสะท้อนกลับได้ LED จึงแสดงสถานะว่าเจอพื้นสีขาวโดยการเปล่งแสงสีแดง แต่ถ้าเมื่อไหร่เจอพื้นสีเข้มหรือตัวหุ่นยนต์ลอยกลางอากาศ แสงก็จะไม่มีการสะท้อนกลับไปยังตัวรับของเซ็นเซอร์ LED จึงไม่แสดงผลเป็นสีแดง



จากภาพด้านบน Serial Monitor แสดงเป็นเลข 0 เมื่อเจอพื้นสีขาว ส่วนรูปด้านล่างแสดงเป็นเลข 1 เมื่อลอยกลางอากาศหรือพื้นสีเข้ม



ในการเขียนโปรแกรมให้รถหุ่นยนต์เดินตามเส้นนั้น จำเป็นต้องใช้เซ็นเซอร์อย่างน้อย 2 ตัว โดยแยกสถานการณ์เป็นดังนี้ สมมติว่าเราใช้พื้นสีขาวหรือสีอ่อนเป็นสนามและใช้เทปพันสายไฟดังภาพเพื่อแปะเป็นเส้นทางให้หุ่นเดินตาม



กำหนดอัลกอริธึมเป็นดังนี้

- เริ่มต้นการทดสอบที่เซ็นเซอร์ซ้ายอยู่ทางด้านซ้ายของแนวเทปพันสายไฟสีดำ เซ็นเซอร์ซ้ายจะอ่านได้เป็น 0

- เซ็นเซอร์กลางอยู่บนแนวเทปพันสายไฟสีดำ ค่าเซ็นเซอร์กลางจะอ่านได้เป็น 1
- เซ็นเซอร์ขวาอยู่ทางด้านขวาของแนวเทปพันสายไฟสีดำ เซ็นเซอร์ขวาจะอ่านได้เป็น 0
- ถ้าค่าจากเซ็นเซอร์ทั้ง 3 เป็นดังข้างบน สั่งให้ล้อรถหุ่นยนต์ทั้งซ้ายและขวาหมุนไปข้างหน้าด้วยความเร็วเท่ากัน
- จากนั้นพิจารณาจากสถานการณ์ต่อไปนี้
    - ถ้าเซ็นเซอร์ซ้ายเป็น 0 เซ็นเซอร์กลางเป็น 0 และเซ็นเซอร์ขวาเป็น 1 แสดงว่ารถหุ่นยนต์กำลังออกนอกแนวเทปพันสายไฟไปทางซ้าย ให้หยุดล้อขวาหรือสั่งให้ล้อขวาวิ่งช้าลง จนกว่าเซ็นเซอร์ซ้ายเป็น 0 เซ็นเซอร์กลางเป็น 1 และเซ็นเซอร์ขวาเป็น 0 เหมือนตอนเริ่มต้น
    - ถ้าเซ็นเซอร์ซ้ายเป็น 1 เซ็นเซอร์กลางเป็น 0 และเซ็นเซอร์ขวาเป็น 0 แสดงว่ารถหุ่นยนต์กำลังออกนอกแนวสายไฟไปทางขวา ให้หยุดล้อซ้ายหรือสั่งให้ล้อซ้ายวิ่งช้าลง จนกว่าเซ็นเซอร์ซ้ายเป็น 0 เซ็นเซอร์กลางเป็น 1 และเซ็นเซอร์ขวาเป็น 0 เหมือนตอนเริ่มต้น
    - ถ้าเซ็นเซอร์ทั้ง 3 อ่านค่าได้เป็น 1 หมดแปลว่าเจอพื้นสีเข้ม หรือเป็น 0 ทั้งหมด อาจจะเป็นเพราะวิ่งเร็วเกินไป ทำให้แรงเฉื่อยมากจนหยุดไม่ทัน อาจจะสั่งให้หยุดและถอยหลังแล้ววิ่งไปหน้าช้า ๆ อีกรอบ เพื่อให้โปรแกรมทำงานได้ทัน

ด้านล่างเป็นตัวอย่างโปรแกรมให้รถหุ่นยนต์เดินตามเส้นอย่างง่าย ตามอัลกอริธึมด้านบน แต่ถ้าเจอสถานการณ์ 0 ทั้งหมดหรือ 1 ทั้งหมด หุ่นยนต์จะหยุด จะต้องยกหุ่นยนต์มาวางบนเส้นเพื่อให้เดินอีกรอบ

```
int LEFT_WHEELS = 2;
int LEFT_WHEELS_SPEED = 5;
int RIGHT_WHEELS = 4;
int RIGHT_WHEELS_SPEED = 6;

int LEFT_SENSOR = 7;
int MID_SENSOR = 8;
int RIGHT_SENSOR = 9;

void car_forward(int left_speed, int right_speed) {
  digitalWrite(LEFT_WHEELS, HIGH);
```

```
  analogWrite(LEFT_WHEELS_SPEED, left_speed);
  digitalWrite(RIGHT_WHEELS, LOW);
  analogWrite(RIGHT_WHEELS_SPEED, right_speed);
}

void setup() {
  pinMode(LEFT_WHEELS, OUTPUT);
  pinMode(LEFT_WHEELS_SPEED, OUTPUT);
  pinMode(RIGHT_WHEELS, OUTPUT);
  pinMode(RIGHT_WHEELS_SPEED, OUTPUT);

  pinMode(LEFT_SENSOR, INPUT_PULLUP);
  pinMode(MID_SENSOR, INPUT_PULLUP);
  pinMode(RIGHT_SENSOR, INPUT_PULLUP);
}

void loop() {
  int left_value(digitalRead(LEFT_SENSOR));
  int mid_value(digitalRead(MID_SENSOR));
  int right_value(digitalRead(RIGHT_SENSOR));

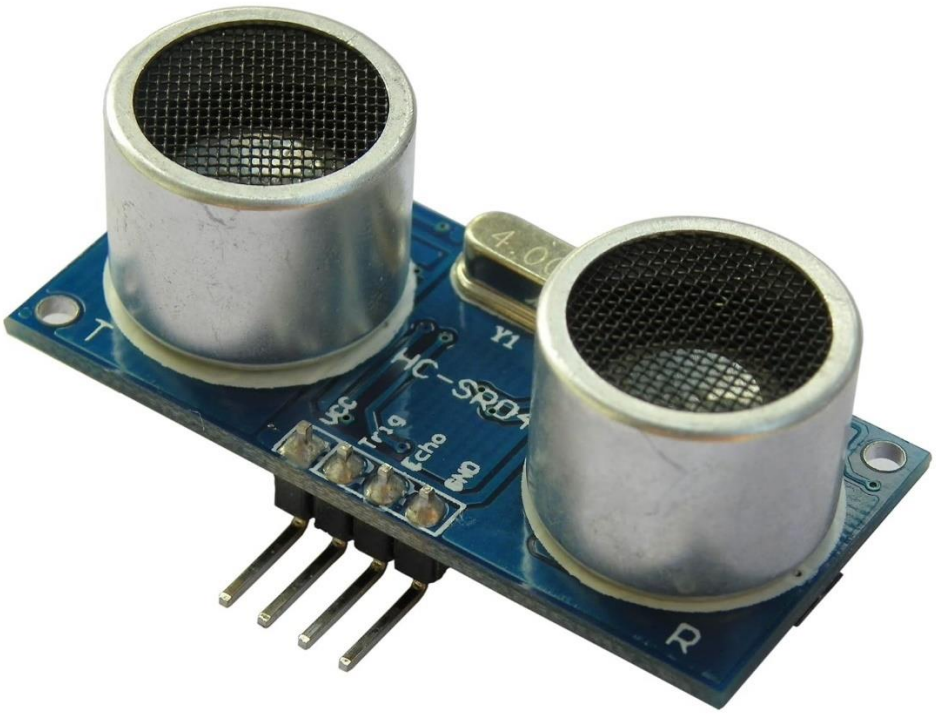
  if (left_value == 0 && mid_value == 1 && right_value == 0) { //OK
    car_forward(120, 120);
  } else if (left_value == 0 && mid_value == 0 && right_value == 1) {
//slight left
    car_forward(10, 120);
  } else if (left_value == 1 && mid_value == 0 && right_value == 0) {
//slight right
    car_forward(120, 10);
  } else { // error choose to stop
    car_forward(0, 0); //stop
```

```
    }
}
```

3. การเขียนโปรแกรมให้หุ่นยนต์เดินหลบหลีกสิ่งกีดขวาง

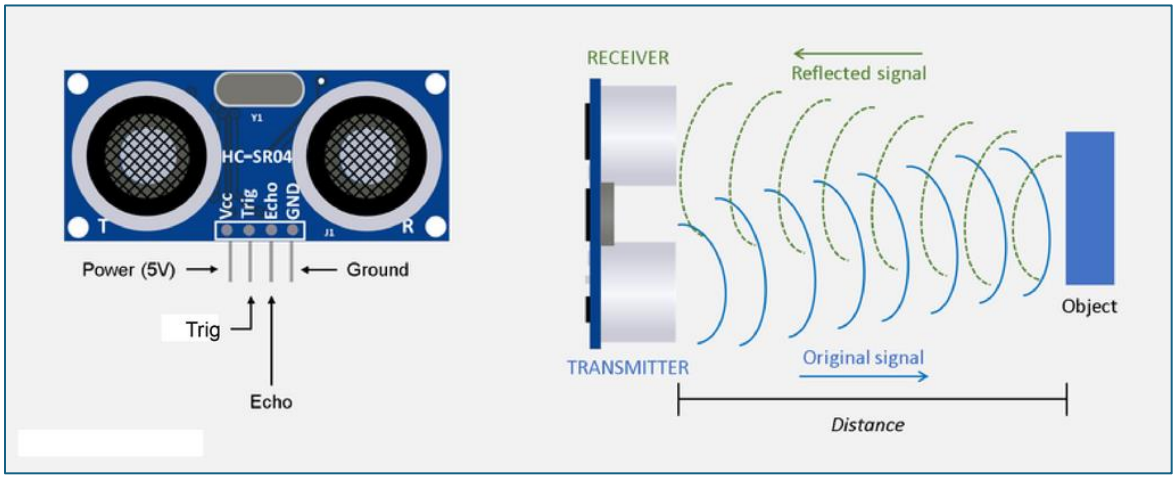
ในการเขียนโปรแกรมควบคุมรถหุ่นยนต์ให้หลบหลีกสิ่งกีดขวางนั้น จะอาศัยเซ็นเซอร์แบบอัลตร้าโซนิคเป็นตัวช่วย หลักการคือมีอุปกรณ์สองตัวทำหน้าที่ในการตรวจจับสิ่งกีดขวาง ตัวแรกทำหน้าที่ส่งเสียงความถี่สูง (สูงเกิน 20000Hz เลยทำให้มนุษย์ไม่สามารถได้ยิน) เมื่อเสียงไปเจอวัตถุ ก็จะสะท้อนกลับมา อุปกรณ์ตัวที่สองจะได้รับเสียงสะท้อน และตอนที่ตัวแรกส่งเสียงออกไป ได้มีการบันทึกเวลาตอนส่งเอาไว้แล้ว และเมื่อตัวที่สองได้รับเสียงสะท้อนกลับมาก็จะตรวจจับเวลาอีกรอบ เมื่อเอาเวลามาหักลบกัน ก็จะได้ระยะเวลาที่เสียงไปกลับ ทำให้สามารถคำนวณระยะทางได้ จากความรู้ที่ว่าเสียงเดินทางด้วยความเร็วประมาณ 340 เมตร/วินาที

จากรูปเซ็นเซอร์อัลตร้าโซนิคด้านล่าง ให้สังเกตตัวอักษรที่เขียน T หมายถึง Transmitter หรือตัวส่งสัญญาณเสียง ส่วนตัวที่เขียนว่า R หมายถึง Receiver หรือตัวรับสัญญาณเสียง



ที่มา: https://m.media-amazon.com/images/I/81g9hfoOqZL._SL1500_.jpg

ในการเขียนโปรแกรมเพื่อตรวจจับสิ่งกีดขวางข้างหน้ารถหุ่นยนต์ด้วยอุลตร้าซาวน์โซนิคเซ็นเซอร์นั้น จำเป็นใช้ digital pin จำนวน 2 pin สำหรับส่งและรับเสียงตามลำดับ ในที่นี้จะใช้ digital pin หมายเลข 12 สำหรับส่งเสียงอัลตร้าโซนิค และ digital pin หมายเลข 13 สำหรับรับเสียงที่สะท้อนกลับ โดยการใช้คำสั่ง pulseIn เพื่อวัดระยะเวลาที่เสียงเดินทาง และเสียงเดินทางประมาณ 58 ไมโครวินาที ต่อ 1 เซนติเมตร



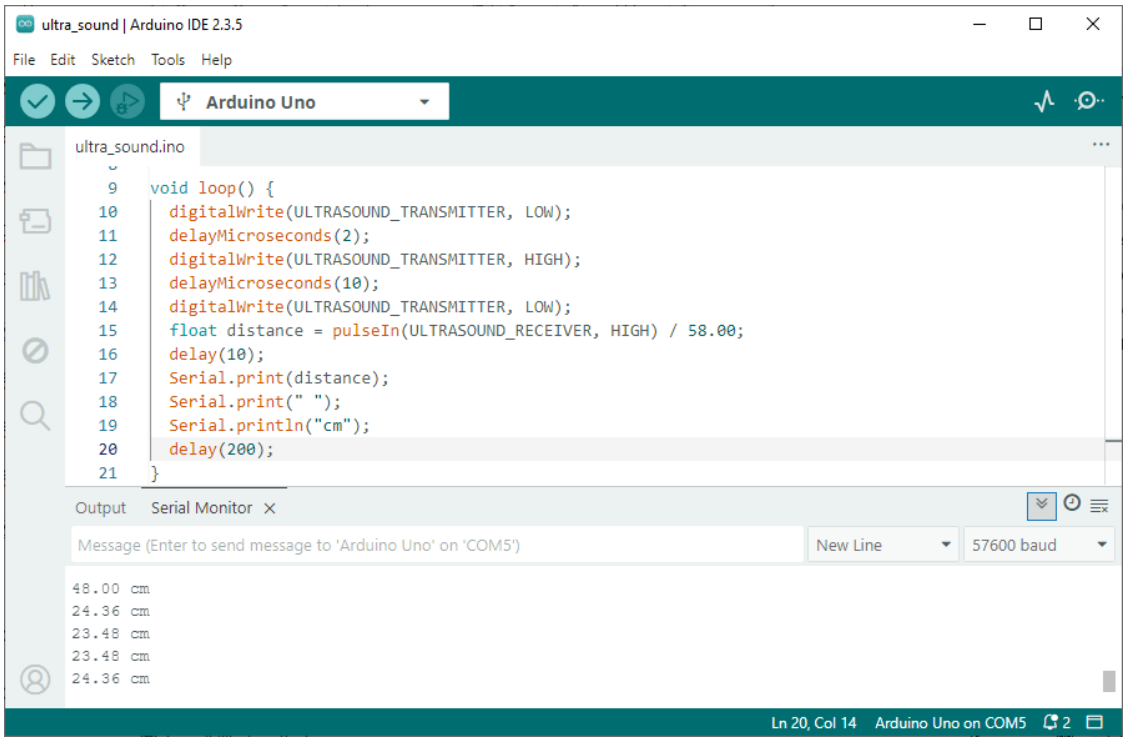
ที่มา: https://content.instructables.com/FJL/J75O/JZX168JW/FJLJ75OJZX168JW.png

```
int ULTRASOUND_TRANSMITTER = 12;
int ULTRASOUND_RECEIVER = 13;
void setup() {
  pinMode(ULTRASOUND_TRANSMITTER, OUTPUT);
  pinMode(ULTRASOUND_RECEIVER, INPUT);
  Serial.begin(57600);
}
void loop() {
  digitalWrite(ULTRASOUND_TRANSMITTER, LOW);
  delayMicroseconds(2);
  digitalWrite(ULTRASOUND_TRANSMITTER, HIGH);
  delayMicroseconds(10);
  digitalWrite(ULTRASOUND_TRANSMITTER, LOW);
  float distance = pulseIn(ULTRASOUND_RECEIVER, HIGH) / 58.00;
```

```
  delay(10);
  Serial.print(distance);
  Serial.print(" ");
  Serial.println("cm");
  delay(200);
}
```

เมื่อ upload โปรแกรมขึ้นไปยังรถหุ่นยนต์แล้ว ให้ทดลองใช้มือขวางด้านหน้าของรถหุ่นยนต์ ใน Serial Monitor ก็จะพบว่ามีการแสดงระยะทางระยะทางระหว่างมือกับหุ่นยนต์เป็นหน่วยเซนติเมตร จากการแสดงระยะทางดังกล่าว จะทำให้สามารถทราบได้ว่าระยะทางระหว่างหุ่นยนต์กับวัตถุที่อยู่ด้านหน้า ห่างเป็นระยะทางกี่เซนติเมตร

ตัวอย่างการเขียนโปรแกรมให้รถหุ่นยนต์หลบหลีกสิ่งกีดขวางนั้น เริ่มจากสั่งให้รถหุ่นยนต์วิ่งไปด้านหน้าก่อน สมมติเจอว่ามีวัตถุขวางทางด้านหน้าในระยะ 50 เซนติเมตร รถหุ่นยนต์ก็จะยังวิ่งตรงไปด้านหน้าได้อีกประมาณ 30 เซนติเมตร แต่เมื่อเหลือ 20 เซนติเมตรหรือน้อยกว่า จะต้องเลี้ยวซ้ายหรือขวาเพื่อหลบ จากนั้นก็ตรวจสอบระยะห่างจากวัตถุกับหุ่นยนต์อีกรอบ ถ้ายังมีระยะน้อยกว่าหรือเท่ากับ 20 เซนติเมตรให้เลี้ยวไปเรื่อย ๆ จนกว่าจะมีระยะมากกว่า 20 เซนติเมตรจึงให้เดินหน้าอีกรอบ ทำแบบนี้ไปเรื่อย ๆ ก็จะไม่ชนสิ่งกีดขวางเลย



ต่อไปนี้เป็นตัวอย่างของการเขียนโปรแกรมหลบหลีกสิ่งกีดขวาง โดยมีหลักการว่าเมื่อไหร่เจอสิ่งกีดขวางในระยะมากกว่า 20 เซนติเมตรให้หุ่นเดินไปข้างหน้าเรื่อย ๆ และเมื่อเจอสิ่งกีดขวางมีระยะน้อยกว่าหรือเท่า 20 เซนติเมตร ให้สุ่มว่าจะเลี้ยวซ้ายหรือขวา โดยการเลี้ยวจะใช้เวลาหมุนซ้ายหรือขวาจากการสุ่มเป็นเวลา 0.5 วินาที จากนั้นตรวจสอบระยะสิ่งกีดอีกครั้ง ถ้าไม่มีสิ่งกีดขวางในระยะ 20 เซนติเมตร ก็จะเดินหน้าอีกรอบ ทำแบบนี้ไปเรื่อย ๆ รถหุ่นยนต์ก็จะไม่ชนสิ่งกีดขวาง ยกเว้นว่าสิ่งกีดขวางนั้นเล็กหรือนุ่มเกินกว่าจะสะท้อนเสียงกลับมาได้

```
int LEFT_WHEELS = 2;
int LEFT_WHEELS_SPEED = 5;
int RIGHT_WHEELS = 4;
int RIGHT_WHEELS_SPEED = 6;
int ULTRASOUND_TRANSMITTER = 12;
int ULTRASOUND_RECEIVER = 13;

void car_forward(int left_speed, int right_speed) {
    digitalWrite(LEFT_WHEELS, HIGH);
    analogWrite(LEFT_WHEELS_SPEED, left_speed);
    digitalWrite(RIGHT_WHEELS, LOW);
    analogWrite(RIGHT_WHEELS_SPEED, right_speed);
}
void car_rotate_left(int speed) {
    digitalWrite(LEFT_WHEELS, LOW);
    analogWrite(LEFT_WHEELS_SPEED, speed);
    digitalWrite(RIGHT_WHEELS, LOW);
    analogWrite(RIGHT_WHEELS_SPEED, speed);
}
void car_rotate_right(int speed) {
    digitalWrite(LEFT_WHEELS, HIGH);
    analogWrite(LEFT_WHEELS_SPEED, speed);
    digitalWrite(RIGHT_WHEELS, HIGH);
    analogWrite(RIGHT_WHEELS_SPEED, speed);
```

```
}
void setup() {
  pinMode(LEFT_WHEELS, OUTPUT);
  pinMode(LEFT_WHEELS_SPEED, OUTPUT);
  pinMode(RIGHT_WHEELS, OUTPUT);
  pinMode(RIGHT_WHEELS_SPEED, OUTPUT);
  pinMode(ULTRASOUND_TRANSMITTER, OUTPUT);
  pinMode(ULTRASOUND_RECEIVER, INPUT);
  Serial.begin(57600);
}
void loop() {
  randomSeed(analogRead(0));
  digitalWrite(ULTRASOUND_TRANSMITTER, LOW);
  delayMicroseconds(2);
  digitalWrite(ULTRASOUND_TRANSMITTER, HIGH);
  delayMicroseconds(10);
  digitalWrite(ULTRASOUND_TRANSMITTER, LOW);
  float distance = pulseIn(ULTRASOUND_RECEIVER, HIGH) / 58.00;
  if (distance < 20.0) {
    int rand_value = random(1, 200);
    if (rand_value > 100) {
      car_rotate_right(100);
    } else {
      car_rotate_left(100);
    }
    delay(500);
  } else {
    car_forward(100, 100);
  }
  delay(100);
}
```

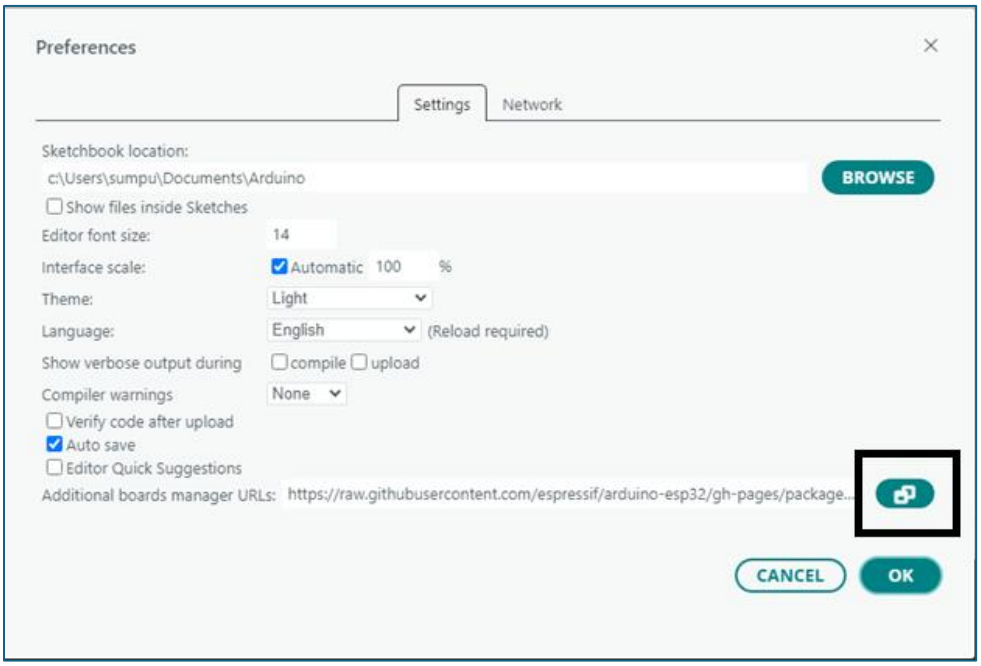
4. การควบคุมรถหุ่นยนต์ทาง WiFi

รถหุ่นยนต์รุ่นนี้มีไมโครคอนโทรลเลอร์จำนวน 2 ตัวคือ ESP32-CAM และ Arduino Uno โดยที่ ESP32-CAM นั้นมีความสามารถหลัก ๆ ดังนี้คือ 1) เชื่อมต่อ WiFi ได้ทั้ง Access Point Mode และ Infrastructure Mode 2) มี Camera ส่วน Arduino Uno นั้นไม่สามารถใช้งาน WiFi ได้ แต่สามารถสั่งงานล้อของหุ่นยนต์ได้ รวมถึงสามารถใช้งาน Ultrasonic Sensor และ Line Tracking Sensor ได้ ดังนั้นเมื่อต้องการควบคุมรถหุ่นยนต์ผ่าน Web Browser ทั้งบนคอมพิวเตอร์และอุปกรณ์เคลื่อน จำเป็นต้องเชื่อมต่อทาง WiFi ไปยัง ESP32-CAM แล้วให้ ESP32-CAM สั่งงานบอร์ด Arduino อีกทอดหนึ่งผ่านคำสั่ง Serial1 ดังนั้นจำเป็นต้องเขียนโปรแกรมบน ESP32-CAM และบน Arduino Uno ไปด้วยกัน

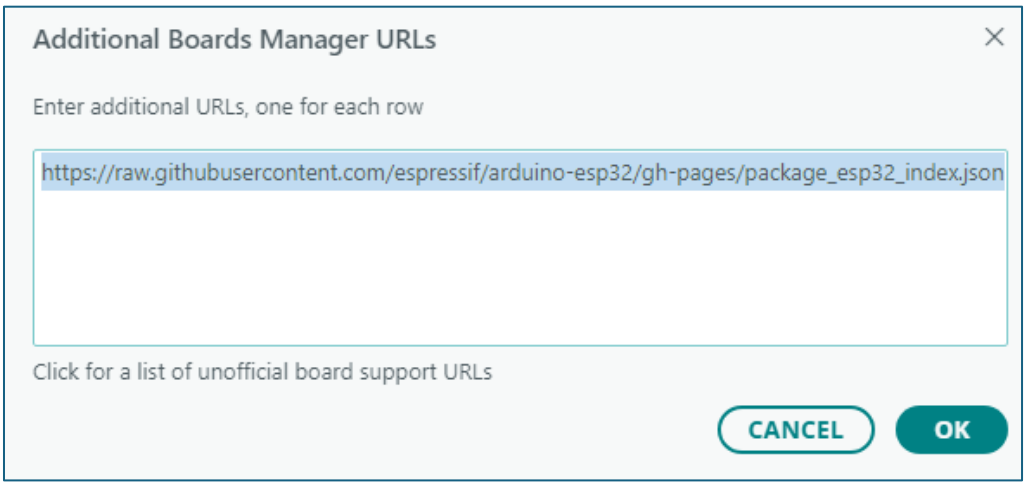
การตั้งค่า Arduino IDE สำหรับเขียนโปรแกรมกับ ESP32-CAM

จริง ๆ แล้ว ESP32-CAM ก็เป็นบอร์ด IoT อีกเวอร์ชันหนึ่งที่ใช้ CHIP ESP32 แต่มีการเพิ่มกล้อง(Camera) เข้าไปด้วย ทำให้สามารถทำตัวเป็น WiFi camera ได้ และยังคงความสามารถทางด้าน IoT เอาไว้ เพียงแต่จะมีจำนวนขาที่สามารถใช้งานอย่างอื่นเหลือน้อย เนื่องจากขาส่วนใหญ่นำไปเชื่อมต่อกับกล้อง

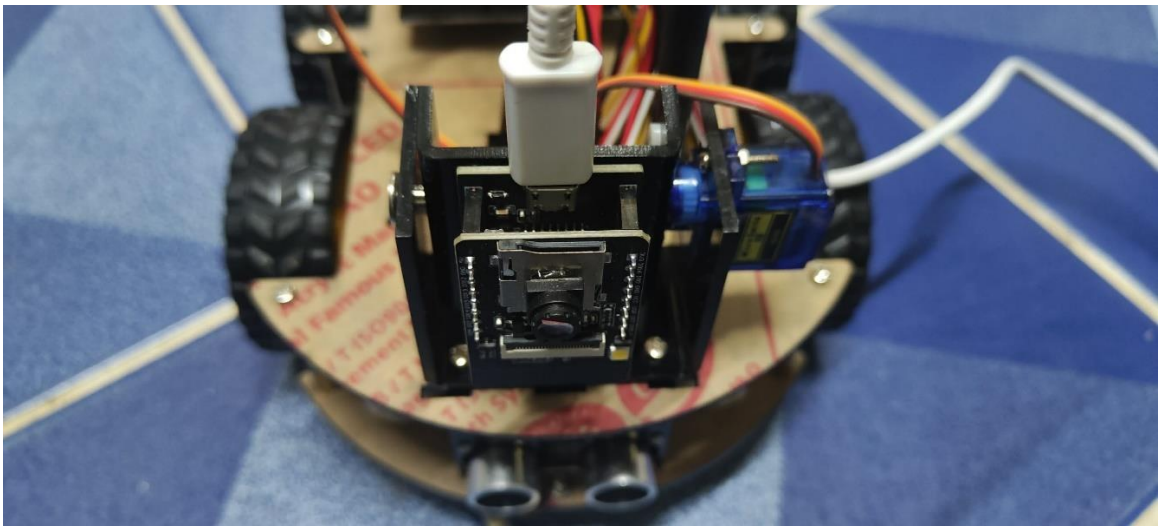
ขั้นแรกเริ่มด้วยการเพิ่ม Addition boards ให้ Arduino IDE รู้จัก ESP32-CAM เสียก่อน โดยการเข้าไปเพิ่ม URL ตรงที่เมนู File->Preferences และเลือกตรง Additional Boards Manager ดังภาพ



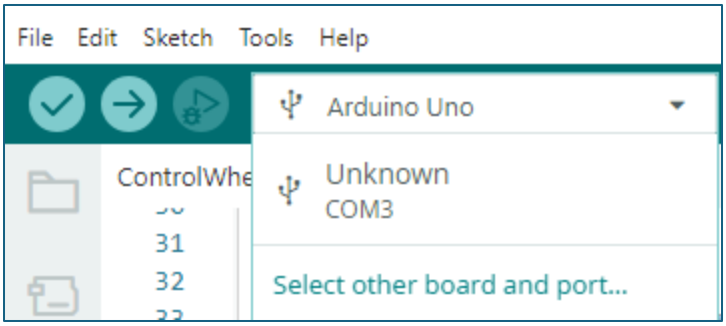
เมื่อคลิ๊กปุ่มที่วงเป็นสี่เหลี่ยมเอาไว้ตามภาพด้านบน ก็จะได้หน้าต่างแบบ dialog ด้านล่าง จากนั้นให้กรอก URL ต่อไปนี้ลงไป https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json จากนั้นคลิ๊ก OK เพื่อบันทึก



หลังจากนั้นให้เชื่อมต่อสาย USB แบบ Micro USB ตรงด้านบนของบอร์ด ESP32-CAM ที่อยู่ด้านหน้าของรถหุ่นยนต์ดังภาพเข้ากับคอมพิวเตอร์

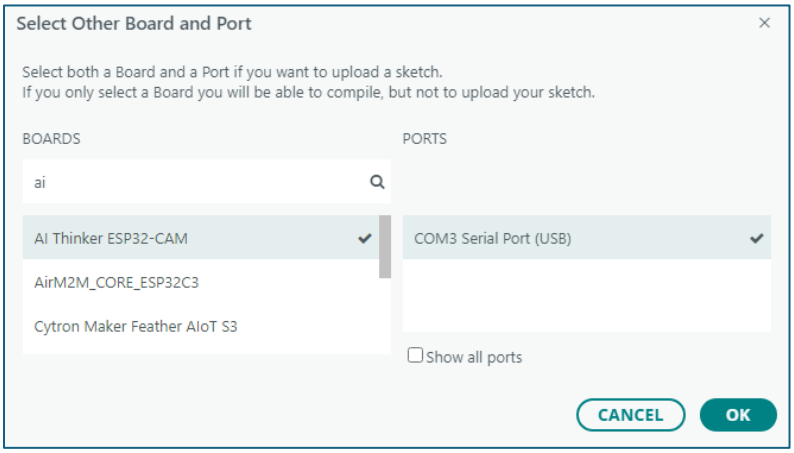


และเมื่อเสียบสาย USB อีกด้านเข้ากับคอมพิวเตอร์ที่เปิด Arduino IDE อยู่ จะแสดงเป็น Unknown COMx โดยที่ x หมายถึงตัวเลข แต่ละเครื่องอาจจะแสดงตัวเลขไม่เหมือนกัน จากนั้นให้คลิ๊ก Select other board and port... ตามด้านล่างของภาพที่ปรากฏในภาพด้านล่าง



ก็จะปรากฏ dialog สำหรับค้นหา Board ให้ค้นหาด้วยคำว่า ai ก็จะปรากฏภาพด้านล่าง โดยที่ด้านซ้ายให้เลือกบอร์ดที่ชื่อว่า AI Thinker ESP32-CAM ส่วนด้านขวา เลือก COMx Serial Port (USB) ให้มีเครื่องถูกทั้งสองด้าน ดังภาพด้านล่าง จากนั้นให้กดปุ่ม OK

จากนี้ต่อไป Arduino IDE ก็พร้อมที่จะใช้สำหรับเขียนโปรแกรมสำหรับบอร์ด ESP32-CAM แล้ว การเขียนโปรแกรมก็จะเหมือนกับการเขียนบน Arduino Uno แทบทุกอย่าง เพียงแต่จะมีบางคำสั่งหรือ pin ที่ไม่เหมือนกัน ในส่วนของ ESP32-CAM จะไม่เน้นการเขียนโปรแกรมมากนัก เพราะค่อนข้างมีความซับซ้อน แต่จะเน้นการปรับเปลี่ยนบางจุดของโปรแกรมที่มีอยู่แล้วให้สามารถใช้งานตามต้องการได้



ด้านล่างเป็นตัวอย่างโปรแกรมที่ใช้ในการทดสอบให้ ESP32-CAM ส่งข้อความคำว่า Hello ไปยัง Arduino Uno โดยโปรแกรมนี้จะ Upload ไปยังบอร์ด ESP32-CAM โดยที่การเชื่อมต่อระหว่างบอร์ด ESP32-CAM บนรถหุ่นยนต์รุ่นนี้จะใช้ digital pin หมายเลข 13 สำหรับการส่งออก (TX) ไปยัง digital pin หมายเลข 0 ของบอร์ด Arduino Uno เพื่อรับข้อมูล (RX) ส่วนในฝั่งรับบนบอร์ด ESP32-CAM นั้นไม่มีการเชื่อมต่อเอาไว้ เนื่องจากไม่เป็นจำเป็นต้องใช้งาน หรือ Arduino ไม่จำเป็นต้องส่งข้อมูลอะไรกลับไปให้ ESP32-CAM จึงกำหนดให้

RX_PIN = -1 เพื่อหลอกให้โปรแกรมทำงานได้ การทำงานของโปรแกรมนี้คือส่งคำว่า “Hello” ออกไปทุก ๆ 0.5 วินาที

```
#define RX_PIN -1
#define TX_PIN 13
void setup() {
  Serial1.begin(115200, SERIAL_8N1, RX_PIN, TX_PIN);
}
void loop() {
  Serial1.println("Hello");
  delay(500);
}
```

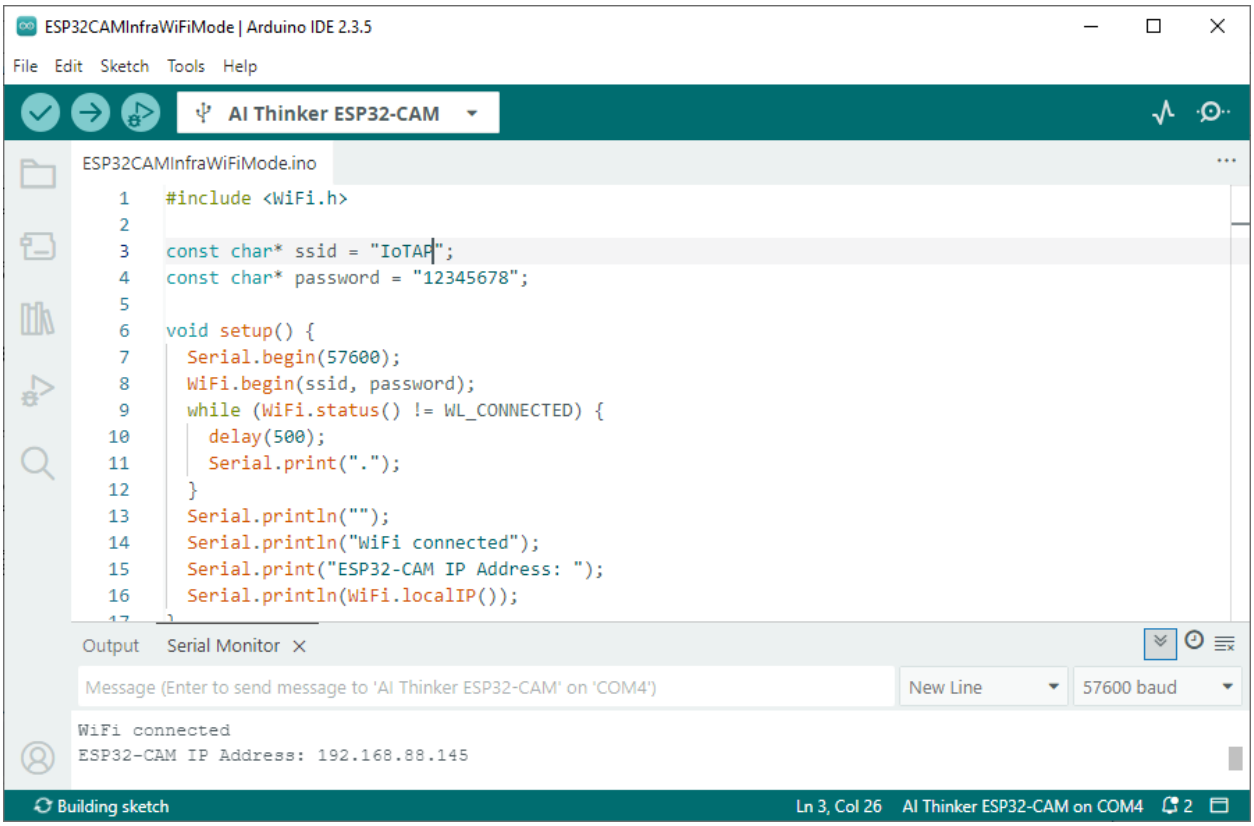
เมื่อ upload โปรแกรมฝั่ง ESP32-CAM สำเร็จแล้ว ต่อไปเป็นการเขียนโปรแกรมเพื่อรับข้อความที่ ESP32-CAM ส่งมาบน Arduino Uno โดยโปรแกรมเป็นไปตามด้านล่าง

```
String temp = "";
void setup() {
  Serial.begin(115200);
}
void loop() {
  temp = "";
  while (Serial.available() > 0) {
     temp = temp + ((char)(Serial.read()));
     delay(2);
  }
  temp.trim();
  if (temp != "") {
     Serial.println(temp);
  }
}
```

จากนั้นให้ upload โปรแกรมนี้เข้าไปยังบอร์ด Arduino Uno แล้วเปิด Serial Monitor ของบอร์ด Arduino Uno โปรแกรมจะแสดงความคำว่า “Hello” บน Serial Monitor ซึ่งเป็นข้อความที่ถูกส่งมาจากบอร์ด ESP32-CAM ทุก ๆ 0.5 วินาที ทั้งนี้อย่าลืมเปิดสวิทช์บนรถหุ่นยนต์ด้วย เพื่อจ่ายกระแสไฟฟ้าให้บอร์ด ESP32-CAM ทำงาน

ต่อ WiFi แบบ Infrastructure Mode

การใช้บอร์ด ESP32-CAM เชื่อมต่อกับเข้าระบบ WiFi ที่มี WiFi Access Point อยู่แล้ว หรือที่เรียกว่า Infrastructure Mode ซึ่งใช้กับระบบที่มี WiFi Access Point ที่เป็นระบบใช้งานแบบทั่วไป ในการทดสอบนี้สมมติว่าที่บ้านมี WiFi Access Point สำหรับใช้งานอินเตอร์เน็ต มี SSID = IoTAP และ password = 12345678 ถ้าแสดงข้อมูลคล้ายภาพด้านล่างแปลว่า ESP32-CAM สามารถต่อกับ WiFi ได้แล้ว เพราะมีการแสดง IP Address ตามด้านล่างของภาพได้



ตัวอย่างโปรแกรมการเขียนโปรแกรมให้ ESP32-CAM ต่อ WiFi ดังข้างล่าง

```
#include <WiFi.h>
```

```
const char* ssid = "IoTAP";
const char* password = "12345678";

void setup() {
  Serial.begin(57600);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi connected");
  Serial.print("ESP32-CAM IP Address: ");
  Serial.println(WiFi.localIP());
}
void loop() {
}
```

การต่อ WiFi แบบ Access Point Mode

ในกรณีที่ไม่มี WiFi Access Point อยู่ก่อน สามารถให้ตัวรถหุ่นยนต์ทำตัวเป็น WiFi Access Point เสียเองก็ได้ โดยการกำหนดชื่อ SSID และ Password ขึ้นมาเอง จากนั้นให้คำสั่ง WiIFi.softAP() แทนที่คำสั่ง WiFi.begin() ดังตัวอย่างโปรแกรมถัดไป

```
#include <WiFi.h>
const char* ssid = "ESP32-CAM_AP";
const char* password = "12345678";
void setup() {
  Serial.begin(57600);
  WiFi.softAP(ssid, password);
  Serial.println("");
  Serial.print("ESP32-CAM IP Address: ");
```

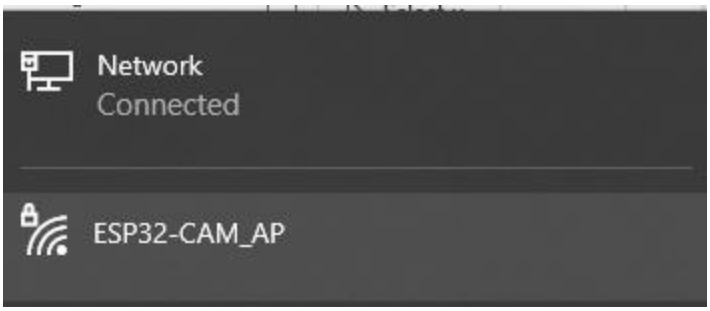
```
  Serial.println(WiFi.softAPIP());
}

void loop() {
}
```

เมื่อ upload โปรแกรมเสร็จแล้ว ให้ลองถอดสาย USB แล้วเสียบใหม่อีกรอบ ถ้าเห็น IP Address 192.168.4.1 เป็นตามแบบด้านล่าง แปลว่าทำถูกต้องแล้ว ส่วนที่แสดงเป็นขยะนั้นไม่ต้องสนใจ



ถ้าลองดูใช้โทรศัพท์มือถือหรือคอมพิวเตอร์ค้นหา WiFi Access Point ก็จะพบ Access Point ที่มีชื่อตามที่ได้ตั้งไว้ในโปรแกรม แสดงเพิ่มขึ้นมาอีก 1 รายการ และสามารถทดลองเชื่อมต่อดูด้วยรหัสผ่านตามที่กำหนดไว้ในโปรแกรมได้ แต่ในขั้นนี้ยังไม่สามารถใช้อย่างอื่นได้นอกจากทดสอบว่าสามารถเชื่อมต่อได้หรือไม่



ในการเข้าไปสั่งงานบอร์ด ESP32-CAM ผ่าน WiFi นั้นทำได้หลายวิธี แต่วิธีที่ได้รับความนิยมมากที่สุดคือทำให้ ESP32-CAM เป็น Web Server จากนั้นสามารถใช้ Web Browser เชื่อมต่อเข้าไปสั่งงาน วิธีนี้เป็นวิธีที่ได้ผลที่สุด เนื่องจากว่าจะไม่ติดปัญหาเรื่องไฟร์วอล และการปิดพอร์ตของ router ต่าง ๆ บนอินเตอร์เน็ต

แต่ในการสร้าง Web Server บน ESP32-CAM และการใช้กล้องบน ESP32-CAM จะไม่ขอลงรายละเอียด เนื่องจากตัวโค้ดมีความซับซ้อนพอสมควรแล้วจะใช้เวลานานและยาก แต่จะใช้วิธีให้แก้โค้ดในจุดที่จะทำให้โปรแกรมสามารถใช้งานได้ ก็คงเพียงพอสำหรับการอบรมครั้งนี้ แต่จะอธิบายหลักการทำงานอย่างง่ายเพื่อเป็นพื้นฐาน

Web Server โดยปกติแล้วจะให้บริการไฟล์ที่มีเนื้อหาข้างในเป็นภาษา HTML/JavaScript/CSS เท่านั้น สาเหตุเพราะว่า Web Browser รู้จักเพียง 2 ภาษาคือ HTML กับ JavaScript รวมถึง Cascading Style Sheet (CSS) หลักการทำงานคือเมื่อป้อน URL เช่น http://192.168.4.1 บน Web Browser แล้วกด Enter URL=> 192.168.4.1จะถูกส่งไปยัง Web Server และมีขั้นตอนดังนี้

- Web Browser ส่ง URL: http://192.168.4.1/ ไปยัง Web Server
- Web Server พิจารณาจากสิ่งที่ได้รับ ในที่นี้ 192.168.4.1 คือชื่อ Server หรือ IP Address ของ Server และสิ่งที่ตามหลังคือเครื่องหมาย (Slash) “/”
- เมื่อ Web Server ได้รับ / แปลว่าต้องไปดูว่ามีการกำหนดไว้ว่าให้หาไฟล์ชื่ออะไรเพื่อส่งกลับไปยัง Web Browser ส่วนใหญ่จะตั้งให้ชี้ไปหาไฟล์ index.html ที่เรามักจะเรียกไฟล์ว่าหน้าแรก
- จากนั้น Web Server ก็อ่านไฟล์ index.html จะส่งชนิดชนิด Content กลับไปบอกว่าเป็น Content ประเภทใด และตามด้วยเนื้อหาของ Content เช่น เป็นแบบ text/html แล้วก็ส่งเนื้อหาที่เป็น html ในไฟล์ ตามหลังไป ซึ่ง Content อาจจะเป็นอะไรก็ได้ เช่น streaming, image, etc.

- เมื่อ Web Browser ได้รับ Content ก็สามารถแปลและแสดงผลให้ผู้ใช้สามารถดูและใช้งานได้

จากขั้นตอนด้านบน จะเห็นได้ว่า Web Brower ไม่ได้รับไฟล์ที่เป็นไฟล์ html จาก Web Server โดยตรง แต่จะได้รับ Content ที่เป็น html หรือสื่ออื่น ๆ ดังนั้นแปลว่า Web Server จะส่งอะไรกลับมาก็ได้ ขอให้เป็นภาษาที่ Web Browser เข้าใจได้ก็พอ ด้วยสาเหตุนี้ Web Server อาจจะใช้ภาษาที่เป็น Server Based อย่างเช่นภาษา php สร้าง Content ขึ้นมา แล้วก็ส่งไปให้ Web Browser ซึ่งก็ไม่ผิดอะไรขอให้อยู่ในรูปแบบภาษาที่ Web Browser เข้าใจได้ก็พอ หรืออาจจะให้ Web Server สร้าง Content ขึ้นเองก็ได้

ดังนั้นเราจึงอาจจะเขียนภาษา HTML ฝังไว้บนโค้ดตอนเขียนโปรแกรมบน ESP32-CAM โดยไม่จำเป็นสร้างเป็นไฟล์แต่อย่างใด เช่น เขียนปุ่มต่าง ๆ ส่วนแสดงผลวีดีโอ ตัวอักษร ซึ่งจะเห็นได้ในโค้ดด้านล่าง

นอกจากไปขอ Content จาก Web Server แล้ว Web Browser ยังสามารถส่งข้อมูลไปหา Web Server ได้เช่นกัน ซึ่งเราจะใช้ประโยชน์จากตรงนี้แหละ เช่น URL http://192.168.4.1/action=forward

ให้พิจารณาตรงคำว่า action=forward (ซึ่งจริง ๆ แล้วหลังจากเครื่องหมาย / จะเป็นคำว่าอะไรก็ได้) เมื่อ Web Server ได้รับคำว่า action=forward ก็เขียนโปรแกรมให้ส่งข้อความว่า “F” ไปยังบอร์ด Arduino Uno แทนที่จะส่ง Content กลับไปยัง Web Browser

เมื่อ Arduino Uno ได้รับข้อความว่า “F” ก็จะสั่งให้ล้อของรถหุ่นยนต์หมุนไปด้านหน้า จากวิธีการข้างบนแปลว่าสั่งงานบน Web Browser เพื่อสั่งให้ ESP32-CAM สั่ง Arduino Uno เพื่อสั่งให้ล้อหมุนอีกทอด ในทำนองเดียวกันก็ทำกับการบังคับรถหุ่นยนต์ในทุกทิศทางแบบเดียวกัน

โดยระบุในโค้ดว่าเมื่อมีการ Click ปุ่ม Forward ก็ Web Browser ก็ให้ส่งคำสั่ง http://192.168.4.1/action=forward ไปยัง Web Server ต่อจากนั้น ESP32-CAM ก็ส่งข้อความว่า “F” ไปบอกบอร์ด Arduino Uno ให้สั่งล้อทั้ง 4 ล้อให้หมุนไปข้างหน้า และถ้าเรา Click ปุ่ม Forward ค้างไว้รถหุ่นยนต์ก็จะวิ่งไปข้างหน้าเรื่อย ๆ แต่เมื่อปล่อยการ Click ปุ่ม Forward ที่ค้างไว้ก่อนหน้า Web Browser ก็จะส่ง http://192.168.4.1/action=stop แปลว่าให้รถหุ่นยนต์หยุดวิ่งนั่นเอง

การส่ง Video Streaming จากกล้องบน ESP32-CAM ไปยัง Web Browser

การดู Video Streaming จากกล้อง สามารถทำได้ผ่าน Web Server บน ESP32-CAM ได้เหมือนกัน เช่น เราอาจะกำหนดในโค้ดว่า เมื่อมีการเรียก URL ที่ตามด้วย /streaming ก็ให้ Web Server ส่งไปบอกว่า Content เป็นแบบ vdo/streaming และตามด้วยเนื้อของ video ที่ได้มาจากกล้อง ให้แก่ Web Browser

โปรแกรมด้านล่างเป็นโปรแกรมที่ได้มีการดัดแปลงจากต้นฉบับ เลยต้องคง credit เอาไว้แก่ผู้เขียนคนแรก ตามหลักของโปรแกรมฟรี โดยโปรแกรมสามารถส่งวีดีโอจากกล้องไปยัง Web Browser ได้ และได้เพิ่มความสามารถในการส่งคำสั่งควบคุมรถหุ่นยนต์ไปยัง Arduino Uno ได้อีกด้วย

```
/*********

  Rui Santos & Sara Santos - Random Nerd Tutorials

  Complete instructions at https://RandomNerdTutorials.com/esp32-cam-
projects-ebook/

  Permission is hereby granted, free of charge, to any person obtaining a
copy of this software and associated documentation files.

  The above copyright notice and this permission notice shall be included in
all copies or substantial portions of the Software.

*********/


#include "esp_camera.h"

#include <WiFi.h>

#include "esp_timer.h"

#include "img_converters.h"

#include "Arduino.h"

#include "fb_gfx.h"

#include "soc/soc.h"             // disable brownout problems

#include "soc/rtc_cntl_reg.h"    // disable brownout problems

#include "esp_http_server.h"


// Replace with your network credentials

const char* ssid = "IoTAP";

const char* password = "12345678";

#define PART_BOUNDARY "123456789000000000000987654321"
```

```
//begin for AI Thinker Model Camera
#define PWDN_GPIO_NUM     32
#define RESET_GPIO_NUM    -1
#define XCLK_GPIO_NUM      0
#define SIOD_GPIO_NUM     26
#define SIOC_GPIO_NUM     27

#define Y9_GPIO_NUM       35
#define Y8_GPIO_NUM       34
#define Y7_GPIO_NUM       39
#define Y6_GPIO_NUM       36
#define Y5_GPIO_NUM       21
#define Y4_GPIO_NUM       19
#define Y3_GPIO_NUM       18
#define Y2_GPIO_NUM        5
#define VSYNC_GPIO_NUM    25
#define HREF_GPIO_NUM     23
#define PCLK_GPIO_NUM     22
//end for AI Thinker Model Camera
#define RX_PIN -1
#define TX_PIN 13

static const char* _STREAM_CONTENT_TYPE = "multipart/x-mixed-
replace;boundary=" PART_BOUNDARY;
static const char* _STREAM_BOUNDARY = "\r\n--" PART_BOUNDARY "\r\n";
static const char* _STREAM_PART = "Content-Type: image/jpeg\r\nContent-
Length: %u\r\n\r\n";

httpd_handle_t camera_httpd = NULL;
httpd_handle_t stream_httpd = NULL;
```

```
static const char PROGMEM INDEX_HTML[] = R"rawliteral(
<html>
  <head>
    <title>ESP32-CAM Robot</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <style>
      body { font-family: Arial; text-align: center; margin:0px auto;
padding-top: 30px;}
      table { margin-left: auto; margin-right: auto; }
      td { padding: 8 px; }
      .button {
        background-color: #2f4468;
        border: none;
        color: white;
        padding: 10px 20px;
        text-align: center;
        text-decoration: none;
        display: inline-block;
        font-size: 18px;
        margin: 6px 3px;
        cursor: pointer;
        -webkit-touch-callout: none;
        -webkit-user-select: none;
        -khtml-user-select: none;
        -moz-user-select: none;
        -ms-user-select: none;
        user-select: none;
        -webkit-tap-highlight-color: rgba(0,0,0,0);
      }
      img {  width: auto ;
```

```
      max-width: 100% ;
      height: auto ;
    }
  </style>
</head>
<body>
  <h1>ESP32-CAM Robot</h1>
  <img src="" id="photo" >
  <table>
    <tr>
      <td colspan="3" align="center">
        <button class="button" onmousedown="sendCommand('forward');"
          ontouchstart="sendCommand('forward');"
          onmouseup="sendCommand('stop');"
          ontouchend="sendCommand('stop');">Forward
        </button>
      </td>
    </tr>
    <tr>
      <td align="center">
        <button class="button" onmousedown="sendCommand('left');"
          ontouchstart="sendCommand('left');"
          onmouseup="sendCommand('stop');"
          ontouchend="sendCommand('stop');">Left
        </button>
      </td>
      <td align="center">
        <button class="button" onmousedown="sendCommand('stop');"
          ontouchstart="sendCommand('stop');">Stop
        </button>
      </td>
```

```
        <td align="center">
          <button class="button" onmousedown="sendCommand('right');"
            ontouchstart="sendCommand('right');"
            onmouseup="sendCommand('stop');"
            ontouchend="sendCommand('stop');">Right
          </button>
        </td>
      </tr>
      <tr>
        <td colspan="3" align="center">
          <button class="button" onmousedown="sendCommand('backward');"
             ontouchstart="sendCommand('backward');"
             onmouseup="sendCommand('stop');"
             ontouchend="sendCommand('stop');">Backward
          </button>
        </td>
      </tr>
    </table>
   <script>
   function sendCommand(x) {
     var xhr = new XMLHttpRequest();
     xhr.open("GET", "/action?go=" + x, true);
     xhr.send();
   }
   window.onload = document.getElementById("photo").src =
window.location.href.slice(0, -1) + ":81/stream";
  </script>
  </body>
</html>
)rawliteral";
```

```
static esp_err_t index_handler(httpd_req_t *req){
  httpd_resp_set_type(req, "text/html");
  return httpd_resp_send(req, (const char *)INDEX_HTML, strlen(INDEX_HTML));
}

static esp_err_t stream_handler(httpd_req_t *req){
  camera_fb_t * fb = NULL;
  esp_err_t res = ESP_OK;
  size_t _jpg_buf_len = 0;
  uint8_t * _jpg_buf = NULL;
  char * part_buf[64];

  res = httpd_resp_set_type(req, _STREAM_CONTENT_TYPE);
  if(res != ESP_OK){
    return res;
  }

  while(true){
    fb = esp_camera_fb_get();
    if (!fb) {
      Serial.println("Camera capture failed");
      res = ESP_FAIL;
    } else {
      if(fb->width > 400){
        if(fb->format != PIXFORMAT_JPEG){
          bool jpeg_converted = frame2jpg(fb, 80, &_jpg_buf, &_jpg_buf_len);
          esp_camera_fb_return(fb);
          fb = NULL;
          if(!jpeg_converted){
            Serial.println("JPEG compression failed");
            res = ESP_FAIL;
```

```
          }
        } else {
          _jpg_buf_len = fb->len;
          _jpg_buf = fb->buf;
        }
      }
    }
    if(res == ESP_OK){
      size_t hlen = snprintf((char *)part_buf, 64, _STREAM_PART,
_jpg_buf_len);
      res = httpd_resp_send_chunk(req, (const char *)part_buf, hlen);
    }
    if(res == ESP_OK){
      res = httpd_resp_send_chunk(req, (const char *)_jpg_buf, _jpg_buf_len);
    }
    if(res == ESP_OK){
      res = httpd_resp_send_chunk(req, _STREAM_BOUNDARY,
strlen(_STREAM_BOUNDARY));
    }
    if(fb){
      esp_camera_fb_return(fb);
      fb = NULL;
      _jpg_buf = NULL;
    } else if(_jpg_buf){
      free(_jpg_buf);
      _jpg_buf = NULL;
    }
    if(res != ESP_OK){
      break;
    }
    //Serial.printf("MJPG: %uB\n",(uint32_t)(_jpg_buf_len));
```

```
  }
  return res;
}

static esp_err_t cmd_handler(httpd_req_t *req){
  char*  buf;
  size_t buf_len;
  char variable[32] = {0,};

  buf_len = httpd_req_get_url_query_len(req) + 1;
  if (buf_len > 1) {
    buf = (char*)malloc(buf_len);
    if(!buf){
      httpd_resp_send_500(req);
      return ESP_FAIL;
    }
    if (httpd_req_get_url_query_str(req, buf, buf_len) == ESP_OK) {
      if (httpd_query_key_value(buf, "go", variable, sizeof(variable)) ==
ESP_OK) {
      } else {
        free(buf);
        httpd_resp_send_404(req);
        return ESP_FAIL;
      }
    } else {
      free(buf);
      httpd_resp_send_404(req);
      return ESP_FAIL;
    }
    free(buf);
  } else {
```

```
  httpd_resp_send_404(req);
  return ESP_FAIL;
}

sensor_t * s = esp_camera_sensor_get();
int res = 0;

if(!strcmp(variable, "forward")) {
  Serial.println("Forward");
  Serial1.println("F");

}
else if(!strcmp(variable, "left")) {
  Serial.println("Left");
  Serial1.println("L");
}
else if(!strcmp(variable, "right")) {
  Serial.println("Right");
  Serial1.println("R");
}
else if(!strcmp(variable, "backward")) {
  Serial.println("Backward");
  Serial1.println("B");
}
else if(!strcmp(variable, "stop")) {
  Serial.println("Stop");
  Serial1.println("S");
}
else {
  res = -1;
}
```

```
  if(res){
    return httpd_resp_send_500(req);
  }

  httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");
  return httpd_resp_send(req, NULL, 0);
}

void startCameraServer(){
  httpd_config_t config = HTTPD_DEFAULT_CONFIG();
  config.server_port = 80;
  httpd_uri_t index_uri = {
    .uri       = "/",
    .method    = HTTP_GET,
    .handler   = index_handler,
    .user_ctx  = NULL
  };

  httpd_uri_t cmd_uri = {
    .uri       = "/action",
    .method    = HTTP_GET,
    .handler   = cmd_handler,
    .user_ctx  = NULL
  };
  httpd_uri_t stream_uri = {
    .uri       = "/stream",
    .method    = HTTP_GET,
    .handler   = stream_handler,
    .user_ctx  = NULL
  };
```

```
  if (httpd_start(&camera_httpd, &config) == ESP_OK) {
    httpd_register_uri_handler(camera_httpd, &index_uri);
    httpd_register_uri_handler(camera_httpd, &cmd_uri);
  }
  config.server_port += 1;
  config.ctrl_port += 1;
  if (httpd_start(&stream_httpd, &config) == ESP_OK) {
    httpd_register_uri_handler(stream_httpd, &stream_uri);
  }
}

void setup() {
  WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0); //disable brownout detector

  Serial.begin(115200);
  Serial.setDebugOutput(false);

  Serial1.begin(115200, SERIAL_8N1, RX_PIN, TX_PIN);

  camera_config_t config;
  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer = LEDC_TIMER_0;
  config.pin_d0 = Y2_GPIO_NUM;
  config.pin_d1 = Y3_GPIO_NUM;
  config.pin_d2 = Y4_GPIO_NUM;
  config.pin_d3 = Y5_GPIO_NUM;
  config.pin_d4 = Y6_GPIO_NUM;
  config.pin_d5 = Y7_GPIO_NUM;
  config.pin_d6 = Y8_GPIO_NUM;
  config.pin_d7 = Y9_GPIO_NUM;
  config.pin_xclk = XCLK_GPIO_NUM;
```

```
config.pin_pclk = PCLK_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_sccb_sda = SIOD_GPIO_NUM;
config.pin_sccb_scl = SIOC_GPIO_NUM;
config.pin_pwdn = PWDN_GPIO_NUM;
config.pin_reset = RESET_GPIO_NUM;
config.xclk_freq_hz = 20000000;
config.pixel_format = PIXFORMAT_JPEG;

if(psramFound()){
  config.frame_size = FRAMESIZE_VGA;
  config.jpeg_quality = 10;
  config.fb_count = 2;
} else {
  config.frame_size = FRAMESIZE_SVGA;
  config.jpeg_quality = 12;
  config.fb_count = 1;
}

// Camera init
esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
  Serial.printf("Camera init failed with error 0x%x", err);
  return;
}
// Wi-Fi connection
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
  delay(500);
  Serial.print(".");
```

```
  }
  Serial.println("");
  Serial.println("WiFi connected");

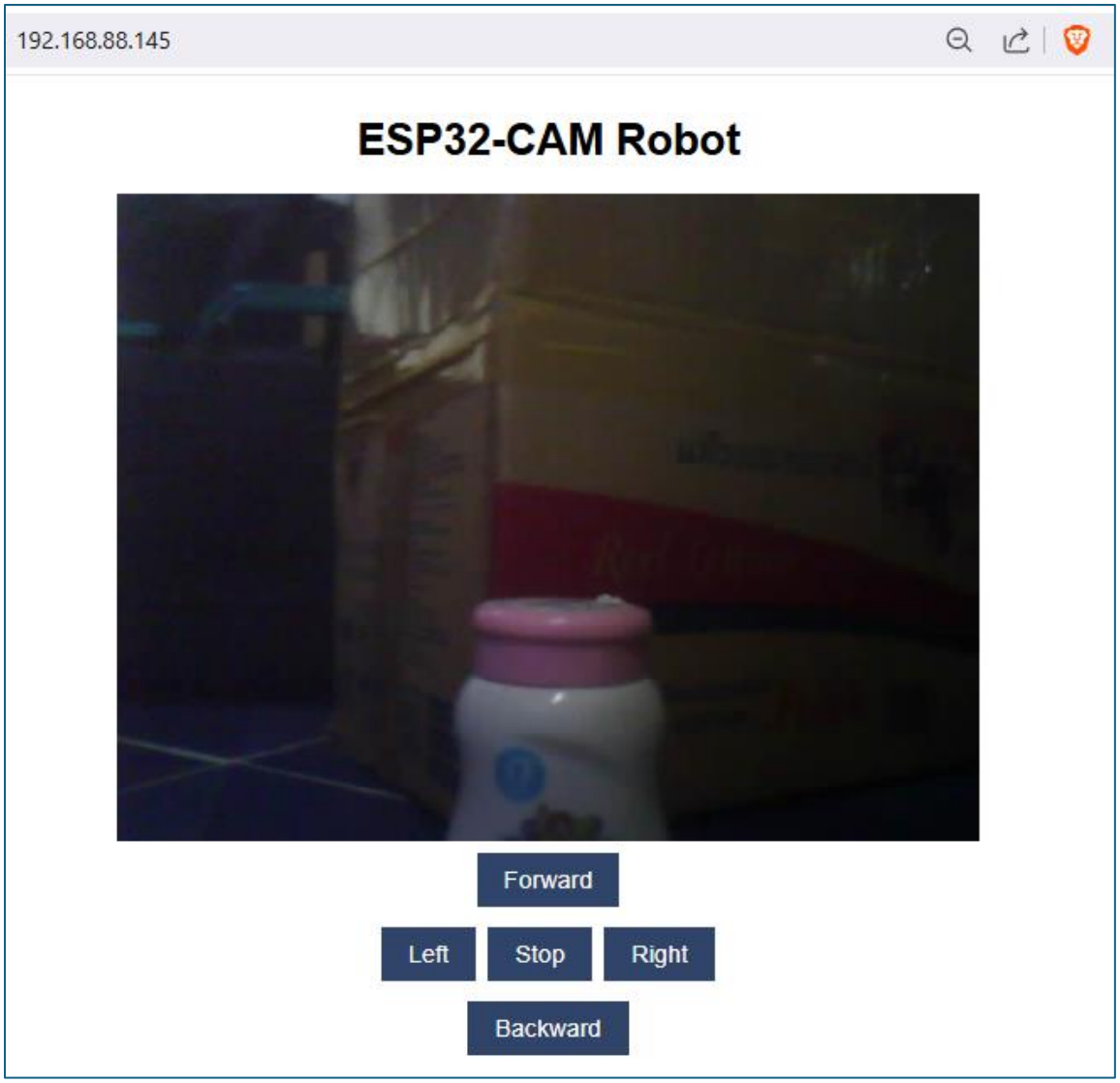
  Serial.print("Camera Stream Ready! Go to: http://");
  Serial.println(WiFi.localIP());

  // Start streaming web server
  startCameraServer();
}

void loop() {
}
```

โดยโปรแกรมข้างบนทำหน้าที่ 3 ประการคือ

1. ส่งโค้ดที่เป็นภาษา html/javascript/css มายัง Web Browser เพื่อให้แสดงผลตามรูปด้านล่าง
2. ส่งภาพที่เป็นภาพเคลื่อนจากกล้องส่งกลับมายัง Web Browser
3. ส่งคำสั่งกลับไปยังเว็บ Web Server เมื่อมีการ Click ปุ่มทิศทาง 5 ปุ่มด้านล่าง



โค้ดหลังจากนี้เป็นโปรแกรมที่ Arduino Uno ใช้สำหรับรับคำสั่งจาก ESP32-CAM เพื่อควบคุมทิศทางของรถหุ่นยนต์

```
int LEFT_WHEELS = 2;
int LEFT_WHEELS_SPEED = 5;
int RIGHT_WHEELS = 4;
int RIGHT_WHEELS_SPEED = 6;


String come_msg;
```

```
void robot_direction(uint8_t left_dir, uint8_t left_speed, uint8_t right_dir,
uint8_t right_speed) {
    digitalWrite(LEFT_WHEELS, left_dir);
    analogWrite(LEFT_WHEELS_SPEED, left_speed);
    digitalWrite(RIGHT_WHEELS, right_dir);
    analogWrite(RIGHT_WHEELS_SPEED, right_speed);
}
void setup() {
    pinMode(LEFT_WHEELS, OUTPUT);
    pinMode(LEFT_WHEELS_SPEED, OUTPUT);
    pinMode(RIGHT_WHEELS, OUTPUT);
    pinMode(RIGHT_WHEELS_SPEED, OUTPUT);

    robot_direction(LOW, 0, LOW, 0); //stop
    Serial.begin(115200);
}

void loop() {
  come_msg = "";
  while (Serial.available() > 0) {
      come_msg = come_msg + ((char)(Serial.read()));
      delay(2);
    }
    come_msg.trim();
    if (come_msg != "") {
      Serial.println(come_msg);
      if (come_msg == "S") { //stop
        robot_direction(LOW, 0, LOW, 0);
      } else if (come_msg == "F") { //forward
        robot_direction(HIGH, 120, LOW, 120);
      } else if (come_msg == "B") { //backward
```

```
        robot_direction(LOW, 120, HIGH, 120);
      } else if (come_msg == "L") { //rotate left
        robot_direction(LOW, 120, LOW, 120);
      } else if (come_msg = "R") { //rotate right
        robot_direction(HIGH, 120, HIGH, 120);
      }
    }
}
```